Native Inference

From grounded judgments to composable proof, computation and meaning

Mettapedia Working Notes

19 October 2026

Abstract

What must a reasoner supply for another reasoner to check an inference? We begin with observations, explicitly admitted assumptions and licensed rule instances. The formal foundation is an effective finitary presentation of judgments, with contexts and binding kept explicit. The open derivations over such a presentation form the free cartesian multicategory on its rule instances; certificate trees are its closed morphisms and shared derivation graphs unfold into them. Translations between kernels are multifunctors, determined by the images of rule instances. Logical work is additive under composition; contraction in a rule image separates tree from graph representations, while extension by definitions changes the multicategory itself. Atomic multi-pattern steps without consumption are graph replay and are independent of the order of independent steps; a consuming split/join process exceeds tree-shaped single-wire-cut composition. A finite evidence checker connects circular strong Kleene specifications to the same proof interface while distinguishing ungroundedness from unfinished checking. Buddhist epistemology and the physical, qualitative, totality and indexical distinctions of scrutability theory motivate explicit evidence boundaries; these philosophical interpretations are distinguished from formal claims. The ledger in Section 14 records names, axioms and status, and Section 15 lists what is open.

Contents

1 What makes a step of reasoning? 2

1.1 Observations, assumptions and checked consequences . . . 3
1.2 A dialogue with Buddhist epistemology . . . 3
1.3 Phenomenal and indexical grounding . . . 3
1.4 From a local step to a proof category . . . 4

2 Scope and relation to the abstract-machine notes 4

3 Circular specifications and finite evidence 5

3.1 Several meanings of an absent answer . . . 6
3.2 Reflection, observation and open-endedness . . . 6

4 The checking ladder 7

4.1 Weighted answers, effects and paid work . . . 8
4.2 Three faces across the ladder . . . 8

5 Rule signatures and the free cartesian multicategory 9

6 Kernels as free multicategories 10

7 Translations as multifunctors **11**

8 Costs: logical work, sharing and extension **12**

9 Atomic multi-pattern steps and schedules **13**

10 Admission and revalidation of retained evidence **14**

11 Module coverage and the LRAT action adapter **14**

12 The costed HOL Light to MM0 translation **15**

13 Dependent judgments, theorem authority and set models **16**

13.1 Finite dependent evidence through the common interfaces 16

13.2 The comparison to sets 17

13.3 Operational logical form and relative checking bases 18

13.4 A computational STT carve-out, with a proved boundary 19

13.5 Conversion policies, proof elaboration and operational fibres 21

13.6 Selected iota roots and checked dependent instantiation 23

13.7 Generated rules, bidirectional evidence and dependent append 25

14 Status ledger **38**

15 Open problems **65**

1 What makes a step of reasoning?

A person sees smoke above a hill and concludes that there is fire. A program matches a stored equation against a subterm and replaces it. A proof assistant applies a lemma to two earlier results. These activities differ in their objects, their sources of authority and their treatment of resources, but they share a question: *what licenses this conclusion from these particular inputs, here?*

Consider the small mathematical inference

$$\frac{e_A : A \qquad e_{A\Rightarrow B} : A \Rightarrow B}{\mathsf{apply}(e_{A\Rightarrow B}, e_A) : B}.$$

The labels e_A and $e_{A\Rightarrow B}$ record evidence, rather than an unexplained assertion that A and $A \Rightarrow B$ are true. Either input may be an assumption, in which case the conclusion remains conditional on it. Alternatively it may have a checked derivation. Recording the dependencies lets another reader distinguish these cases. Modus ponens is one possible licensed rule; it is not built into every rewrite language or every model of reasoning considered here.

The corresponding rewrite example is

$$(\mathsf{add}\ \mathsf{Z}\ n) \longrightarrow n, \qquad \mathsf{S}(\mathsf{add}\ \mathsf{Z}\ \mathsf{Z}) \longrightarrow \mathsf{S}(\mathsf{Z}).$$

A fully specified recipe cites the admitted rule, the inner site, the substitution $n := \mathsf{Z}$ and any required side evidence. The checker replays those choices. The rule does not automatically apply under every constructor: the selected language must license the surrounding context. An equality judgment, a directed execution step and a claim about an evaluator's complete result multiset are different conclusions.

This is a foundation for checking licensed inferences, not a claim that all reasoning is deduction. Induction, abduction, analogy and proof search suggest claims or revise assumptions. Their own presentations may license probabilistic updates or defeasible conclusions rather than truth-preserving deduction. A checked Bayesian update, for example, certifies the stated calculation under its model and observations; a high resulting probability is not a deductive proof of the event. The common floor asks for the contract of each step.

1.1 Observations, assumptions and checked consequences

An observation enters a formal account through an interface: a sensor report, a human assertion, a database snapshot, or a chosen mathematical model. Checking a consequence of that report does not check that the sensor was accurate. A certificate can establish

$$\Gamma, \mathsf{Report}(i, o) \vdash C$$

while leaving the reliability and interpretation of the report as explicit premises. This is useful even when a premise is defeasible: changing it identifies which retained conclusions need revalidation.

We therefore separate three obligations. *Admission* states which assumptions, observations and declarations are available. *Inference checking* validates a finite composition under those admissions. *Semantic soundness* relates the admitted rules and their consequences to a stated interpretation. None of these obligations is discharged by merely printing a theorem-shaped atom. A sealed theorem value can enforce the construction discipline inside an implementation; an exported certificate permits independent replay. Their logical comparison is separate from the runtime's unforgeability and lifetime guarantees.

1.2 A dialogue with Buddhist epistemology

The school of Dignaga and Dharmakirti distinguishes perception from inference, and inference for oneself from the presentation of an inference to another. Dharmakirti's *Nyayabindu* develops inferential knowledge through a reason whose relevant characteristics must be established[1]. The familiar smoke/fire example makes the distinction practical: recognizing smoke at this hill and licensing the connection to fire are different tasks. A formal derivation records the connection and the cited instance; it cannot manufacture reliable perception from the appearance of a valid derivation.

This motivates a division between finding a reason and presenting a checkable reason. The connection to certificate checking is architectural, not an identity between Buddhist epistemology and a modern proof calculus. In particular, inference as valid cognition includes epistemic demands that the replay of a formal certificate does not settle. The companion *Formal Buddhist Logic* studies precise readings of valid reasons, non-perception, exclusion and four-corner evidence separately.

One consequence matters immediately: failure to observe is not, by itself, evidence of absence. A negative conclusion requires a premise such as adequate coverage, observability or an exhaustive finite check. Similarly, failure to find a proof is not a proof of negation. Four-corner evidence, where positive and negative support can both occur or both be absent, is useful for some knowledge stores. It is not automatically the logic of every native kernel, and neither four-corner evidence nor strong Kleene partial truth is simply identified with the Buddhist fourfold analysis.

1.3 Phenomenal and indexical grounding

Chalmers's scrutability programme distinguishes physical information P, phenomenal information Q, totality information T and indexical information I[2, 3]. Indexical information supplies a location within an account; the totality component addresses what the account excludes. These distinctions

help ask what a formal reasoning system has been given, but the scrutability thesis does not provide an executable kernel.

Our use is deliberately open-ended. Let $i : I$ specify an agent, stage or location; let $p : P(i)$ specify publicly represented conditions; and let $Q(i, p)$ record further qualitative distinctions. The indexed total space

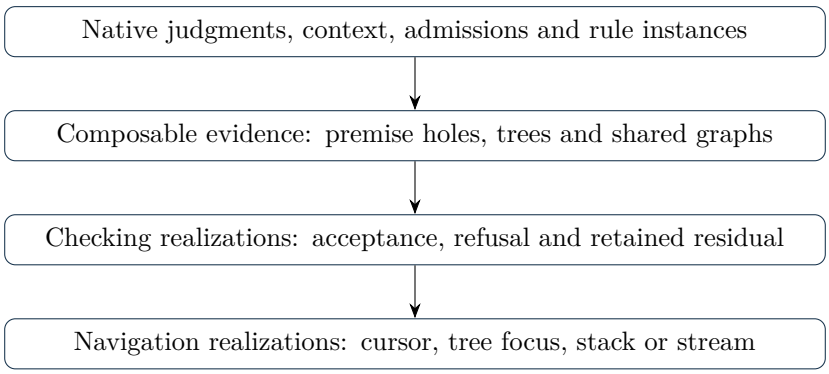
$$\mathcal{E} = \sum_{i:I} \sum_{p:P(i)} Q(i, p)$$

keeps the coordinates distinct. It does not assert that these families exhaust possible experience or that qualitative information is recovered from public data. Replacing the physical vocabulary by abstract laws A gives an $A/Q/I$ variant of this descriptive scheme; it is not a renamed assertion of Chalmers's full $P/Q/T/I$ thesis. A local coverage certificate may authorize a particular negative conclusion without committing to a global "that's all" claim about the world.

The immediate grounding for a human reasoner may include phenomenal experience. A machine account instead requires a specified observation interface; this paper does not establish that the interface is conscious or that recording an experience report captures the experience itself. The operational, intensional and extensional faces of inference are another distinction: an execution, its retained proof evidence and its interpretation. Their comparison maps are mathematical obligations. The three faces are not identified term for term with P, Q and I; in particular a proof's meaning in a Henkin model is not thereby a phenomenal state.

1.4 From a local step to a proof category

A rule has typed input slots and a typed output. Filling its slots with derivations constructs a larger derivation. Filling slots in stages or all at once agrees by associativity; a premise hole is an identity. Multiple input slots lead to a multicategory. A unary rewrite sequence gives the path-category case. This is the elementary categorical content of proof composition, rather than a demand that every language be encoded in one privileged logic.



These are dependency levels, not successive authorities. A cursor's current focus can be made observable, like a reader's position in a proof, but a source calculus need not contain that position. Conversely a language that makes scheduling or resource ownership observable needs a realization theorem preserving precisely those observations. The model should follow those semantics rather than the convenience of a particular storage backend.

2 Scope and relation to the abstract-machine notes

The companion notes *Abstract Machines* (Section 13, "Proof machines: scoped, licensed composition") describe how source judgments, composable evidence, checking realizations and navigation adapters are layered, and which checking machines realize which kernels. This note does not repeat

that material. It supplies the mathematical structure of the evidence layer: what composition of derivations is, how translations act on it, what costs are attached to it, and how atomic multi-premise execution relates to it.

Section map.

Section	Content
3	Circular specifications, finite evidence and semantic partiality
4	The checking ladder and its distinct comparison axes
5	Rule signatures, open derivations and freeness
6	Kernels as free multicategories: trees, graphs and authored calculi
7	Translations as multifunctors
8	Costs: logical work, sharing (SHARE) and extension (EXTEND)
9	Atomic multi-pattern steps and schedules
10	Admission and revalidation of retained evidence
11	Module coverage and the LRAT action adapter
12	The costed HOL Light to MM0 translation
13	Dependent judgments, theorem authority and set models
14	Status ledger
15	Open problems

3 Circular specifications and finite evidence

Circular syntax and circular justification are different. A finite collection of named propositions can contain a reference cycle, while a particular justification for one of its conclusions is still a finite, well-founded tree. For example, if a grounded fact G is true, then $L \vee G$ is true in strong Kleene semantics even when L refers negatively to itself. The evidence uses G; it does not need a proof of L.

Let each judgment $j : J$ have a specification $\mathsf{spec}(j)$ built from base facts, truth references, negation, conjunction and disjunction. A valuation is a pair (H, F) of positive and negative extensions, ordered by inclusion in both coordinates. Strong Kleene evaluation induces a monotone jump $\Phi(H, F)$. Starting with $(\varnothing, \varnothing)$ selects its least fixed point (H_*, F_*)[4]. With consistent base facts the two extensions are disjoint. They need not cover J.

The liar $L = \neg\mathsf{truth}(L)$ receives neither polarity. The truth-teller $U = \mathsf{truth}(U)$ also receives neither in the least fixed point, although other fixed points can make it true or false. Merely checking that an offered valuation is a fixed point therefore does not establish groundedness.

Theorem 3.1 (Grounded evidence as a native proof profile)**.** *For the existing strong Kleene specification system with decidable base admission and decidable equality of judgment and base labels, ten local rule codes give an exact rule-witness interface. A judgment j has a positive or negative grounded verdict exactly when there exists an accepted finite certificate for its specification at that polarity. Equivalently, that signed specification has a closed proof in the free multicategory of the interface. These statements do not require J finite.*

Lean. `KripkeTruth.Checking.interface`, `Checking.check_sound`, `Checking.coverage`, `Checking.judgment_coverage`, `Checking.closed_proof_iff_grounded`

The local rules are the source evidence constructors: admitted positive and negative base leaves; explicit unfolding at a truth reference; polarity reversal at negation; two positive premises for conjunction and either negative premise to refute it; either positive premise for disjunction and two negative premises to refute it. There is no rule that establishes a judgment merely by citing its own label. Generic tree and graph checking apply without a new account of proof composition. This is one particular partial-truth logic hosted by the framework, not a change of every kernel to a three-valued object logic.

Theorem 3.2 (Finite decision agrees with the certificate authority)**.** *Under these assumptions, if J is finite, iteration from the empty valuation reaches (H_*, F_*) after at most $2|J|$ jumps. The implemented Boolean-pair iteration returns a polarity exactly when some finite certificate for that polarity is accepted.*

Lean. `KripkeTruth.System.stage_eq_least`, `System.groundedDecide_spec`, `Checking.finite_decision_iff_certificate`

The bound counts semantic jumps. It neither makes a jump constant time nor proves that the current recursive evaluator avoids repeated work. It depends on a finite judgment domain and decidable base admission. It is not a bound for an unrestricted self-referential truth language with quantifiers.

Example 3.3 (Completion is not bivalence)**.** The finite example computes the twice-ascribed fact as positive, a conjunction with a grounded false component as negative, and the liar and truth-teller as undetermined. The completed liar computation is not unfinished work. Conversely the initially unknown fact becomes true at a later stage, so an intermediate unknown value does not establish ungroundedness. The fact certificate with a forged negative claim is refused, and a self-reference with no supplied premise is refused. The liar has no finite certificate for either polarity.

Lean. `KripkeTruth.Example.decided_verdicts`, `Checking.Controls.fact_accepts`, `Checking.Controls.forged_false_refuses`, `Checking.Controls.absent_premise_refuses`, `Checking.Controls.false_base_leaf_refuses`, `Checking.Controls.cyclic_graph_refuses`, `Checking.Controls.liar_has_no_certificate`

3.1 Several meanings of an absent answer

Outcome	What has been established
Accepted certificate	This evidence checks for this claim under these admissions.
Refused certificate	This candidate fails checking; another candidate may prove the claim.
Negative evidence	A stated negative judgment, or an obstruction under the selected authority.
Grounded undetermined	Neither polarity belongs to the selected least fixed point.
Unfinished check	A retained residual remains; no final verdict follows.
Outside an effective profile	The selected checking method has no authority for this request.
Conflicting reports	A knowledge store has both supports; the chosen logic must specify their consequence.

The shared authority algebra already distinguishes evidence, obstruction, fragment boundaries and restartable resource frontiers. The partial-truth profile gives one mathematically stable meaning to a boundary. It must not turn a timeout, a missing proof or an untrusted report into a falsehood. Nor should the three-valued semantic result replace the richer execution status. Both coordinates can be retained.

Lean. `KripkeTruth.Checking.finite_undetermined_iff_no_certificates`

3.2 Reflection, observation and open-endedness

Self-reference imposes a precise local obstruction: when the liar equation is required, no total Boolean fixed-point valuation satisfies the system. This is not a theorem that every language containing a truth symbol is inconsistent. Gödel incompleteness is a different obstruction: an appropriate consistent, effective arithmetical theory cannot capture every truth of its intended standard model. Reflection levels record which theory a claim concerns; they do not grant an unqualified rule asserting the current theory's own soundness. The existing consistency-tower results are parameterized by their arithmetical hypotheses, which must be instantiated before a concrete base theory is claimed.

Model observations have their own limitations. Indistinguishability by the selected formulas is weaker than isomorphism; downward Lowenheim–Skolem requires the relevant language and parameter cardinal bounds. First-order theories can be categorical at a specified infinite cardinal, so the finite-model observation cannot be generalized to prohibit such categoricity. These model-theoretic qualifications do not alter the local rule checker.

Neither hypersets nor a terminal coalgebra are required for the evidence profile above. A final coalgebra is final for a specified functor in a specified category, not an automatic universe containing every possible logic. Likewise, observer-relative indistinguishability does not justify substitution in arbitrary contexts: a replacement theorem must establish the congruence or naturality law for the contexts actually permitted.

4 The checking ladder

Tier zero names a checking discipline: supply finite evidence for licensed rule applications, with explicit dependencies and enough local choices for a terminating checker. It does not name a weakest axiom system, a unique primitive basis, a universal notion of truth, or a promise of polynomial checking for every possible encoding. A powerful set theory and a modest rewrite calculus can both have this discipline.

Presentation	Supplied evidence	Principal checking obligation
Explicit rule replay	Rule instances, premises, substitutions and side witnesses	Replay each licensed composition.
Retained sharing	A dependency graph and selected root	Validate references and local instances without unfolding shared premises.
Compressed computation	A strategy, fuel and optional checkpoints	Reproduce the specified computation and its endpoint.
Theory extension	Declarations, definitions and admission evidence	State conservativity or new assumptions and revalidate reuse.
Reconstruction	Untrusted tactic or prover output	Produce a certificate that the destination independently checks.

This is a family of comparisons rather than one total ordering. Logical strength, source expressiveness, certificate size, checking time, search power and trusted implementation are separate axes. A proof-size translation does not by itself prove a runtime bound. A faithful map on existing proofs need not reflect the existence of proofs in its target. The intuitionistic-to-classical inclusion below is a concrete example.

Fuel illustrates both the benefit and the price of compression. If a specified machine accepts after n reduction steps, supplying n makes bounded replay decidable and reproduces that successful run. A binary count needs only logarithmically many bits, but replay can still take n steps. Checkpoints need justification between them; an explicit trace writes more information. A trace's linear number of local checks is not automatically linear bit-time if those local checks manipulate large substitutions or expressions.

The library already formalizes fuelled conversion evidence, explicit equality recipes and reproducible strategy hints. It also keeps the Lean4Lean algorithmic checker separate from ideal typing and conversion. More fuel recovers a successful finite run; it cannot make an incomplete algorithm complete for a larger ideal relation. The present executable Lean-related instance is a specified head-conversion profile, not a proof that the full Lean kernel has been reconstructed. The Lean4Less boundary records this distinction rather than treating implementation heartbeats as portable reduction counts.

The comparison target matters more than the implementation language. McTT proves soundness and completeness of an extracted checker for its specified Martin-Lof type theory[9]. The obser-

vational theory TT_{obs} has a normalizing reduction and decidable conversion and type checking for its own rules[10]. These are useful reference profiles; neither result is a completeness theorem for Lean's different judgment, nor does implementing a checker in Rocq identify its object theory with Rocq's full calculus. Runtime efficiency requires a separate measurement.

4.1 Weighted answers, effects and paid work

A useful companion model of an effectful computation is

$$\mathsf{Spec}(K, C, W, X) = W \longrightarrow \mathsf{Ans}_K(X \times W \times C), \qquad \mathsf{Ans}_K(Y) = Y \to_{\mathrm{fin}} K.$$

Here K is a commutative semiring of answer annotations, C a commutative monoid of branch costs, and W a world or space. Sequencing combines weights, threads each branch's state and accumulates its cost. This supplies one lawful model, not a complete semantics of every language that returns several answers. Reads and writes need not commute. A weight is not automatically a probability, a degree of truth or an unresolved scheduler choice. In particular the existing no-distributive-law result concerns probability distributions over finite sets with specified naturality and unit laws; it is not a prohibition on every combination of probabilistic and nondeterministic operations.

Lean. `LayerEffects.Spec.bind_assoc`, `LayerEffects.Controls.put_get_not_comm`, `NondeterminismProbability.no_distributiveLaw`

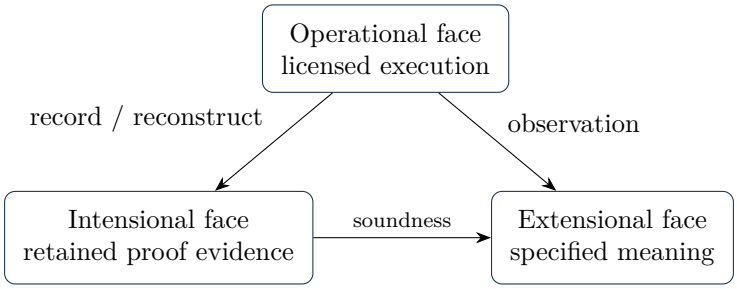
Two proved information-loss boundaries matter for inference. Charging a branch and then failing leaves no successful answer carrying that charge; the total paid work cannot in general be recovered from the surviving answers. It needs an execution account with its own observation contract. Likewise marginal positive/negative counts for A and B do not determine counts for $A \wedge B$ on the same worlds. Cartesian pairing of sample bags has a lawful product, but that product does not recover unknown dependence. A proof checker can validate either stated calculation without silently equating their meanings.

Lean. `LayerEffects.Spec.no_failed_cost_recovery`, `EvidenceSpaces.no_same_world_conjunction_from_marginals`

These models belong downstream of the rule and evidence layer. A categorical completion such as $\mathrm{Ind}(\mathcal{C})$, or a dependent type theory, requires its own objects, judgments and comparison theorems; neither follows from the displayed computation type alone. Similarly, a checked absence frontier proves a negative claim relative to its licensed future relation. Identifying that claim with a guest logic's internal negation requires a separate interpretation theorem.

4.2 Three faces across the ladder

The operational, retained-proof and semantic faces persist across the ladder. An execution explains how checking proceeded, a proof records what licensed the claim, and a semantic map explains what the claim means. A compiler or optimization must name which face it preserves, which it reflects and what information its observation forgets.



For HOL, one extensional judgment is a *semantic sequent* $\Gamma \models_M A$: every admissible assignment in model M satisfying the assumptions in Γ satisfies A. A Henkin model specifies the domains of

quantification, including which function values are available. Soundness connects proof rules to this satisfaction relation; it does not identify the model with the proof's syntax. Different proofs and execution orders can have one semantic sequent. The arrows in the diagram require their stated contracts and generally cannot be inverted.

5 Rule signatures and the free cartesian multicategory

Definition 5.1 (Cartesian multicategory)**.** A cartesian multicategory over a type of sorts Σ assigns to each finite context $\Gamma : \mathrm{Fin}\, n \to \Sigma$ and sort b a type $\mathrm{Hom}(\Gamma; b)$, with variables $\mathrm{var}_i : \mathrm{Hom}(\Gamma; \Gamma_i)$ and simultaneous substitution $\mathsf{subst} : \mathrm{Hom}(\Gamma; b) \times \prod_i \mathrm{Hom}(\Delta; \Gamma_i) \to \mathrm{Hom}(\Delta; b)$, satisfying $\mathrm{var}_i[\sigma] = \sigma_i$, $t[\mathrm{var}] = t$ and $t[\sigma][\tau] = t[\sigma[\tau]]$. A multifunctor over a map of sorts $f : \Sigma \to \Sigma'$ sends $\mathrm{Hom}(\Gamma; b)$ to $\mathrm{Hom}(f \circ \Gamma; fb)$, preserving variables and substitution.

Lean. `CartesianMulticategory`, `CartesianMulticategory.Functor`

Reindexing contexts by composition makes multifunctors compose without transport; the list-indexed `MultiSortedClone` is the restriction to contexts of the form ℓ.get. This is the many-sorted abstract clone presentation; for the correspondence between clones, cartesian multicategories and their internal languages see Saville, *Clones, closed categories, and combinatory logic*[7], which also recalls the multicategories introduced by Lambek in *Deductive systems and categories II*[5]. Lambek and Scott, *Introduction to Higher Order Categorical Logic*, treat the unary deductive systems from which this generalizes.

Proposition 5.2 (Multifunctors form a category)**.** *Identity and composition of multifunctors satisfy the unit and associativity laws. A multifunctor carries closed multimorphisms to closed multimorphisms.*

Lean. `CartesianMulticategory.Functor.id_comp`, `comp_id`, `comp_assoc`, `Functor.inhabited`

Definition 5.3 (Rule signature, open derivations)**.** A rule signature S assigns to each premise list $p_1 \cdots p_k$ and conclusion c a type $S(p; c)$ of rule instances. Open derivations $\mathsf{Term}_S(\Gamma; b)$ are generated by premise holes var_i and nodes $r(t_1, \ldots, t_k)$ with $r \in S(p; c)$ and $t_j \in \mathsf{Term}_S(\Gamma; p_j)$. Substitution plugs derivations into holes. The generator of r is $\mathrm{gen}(r) = r(\mathrm{var}_1, \ldots, \mathrm{var}_k)$.

Lean. `RuleSignature`, `RuleSignature.Term`, `Term.subst`, `RuleSignature.free`, `RuleSignature.gen`

Theorem 5.4 (Freeness)**.** *Open derivations with substitution form a cartesian multicategory* $\mathsf{Free}(S)$*. For every cartesian multicategory M and map of sorts f, restriction to generators is an equivalence between multifunctors* $\mathsf{Free}(S) \to M$ *over f and interpretations*

$$\textstyle\prod_{p,c}\ S(p; c) \longrightarrow \mathrm{Hom}_M(f \circ p;\ fc).$$

Lean. `RuleSignature.universal`, `RuleSignature.functor_eq_of_gen`, `RuleSignature.lift_gen`, `Term.lift_subst`

Theorem 5.5 (Derivability)**.** *A sort is derivable in* $\mathsf{Free}(S)$*, i.e. has a closed derivation, exactly when it lies in the inductive closure of the rule predicate $\exists r \in S(p; c)$. Every multifunctor between free multicategories preserves derivability.*

Lean. `RuleSignature.derivable_iff_derives`, `RuleSignature.Derivable.map`

Proposition 5.6 (Structural rules)**.** *Holes may be discarded or reused, so* $\mathsf{Free}(S)$ *is cartesian; two holes of the same sort remain distinct derivations. Generators are planar (each premise used once, in order), and every closed derivation is planar.*

Lean. `Term.duplicate_holes_distinct`, `Term.gen_linear`, `Term.closed_linear`

Here the holes are metalevel proof dependencies for the fixed rule signature. This construction does not authorize copying an object-level linear resource or a capability. Such constraints must already be present in the judgment, the allowed instances or a resource-sensitive composition. Planarity of these closed terms is not an equivalence of arbitrary linear, symmetric and cartesian object logics.

Theorem 5.7 (Schematic level: instance actions)**.** *If a map of sorts carries every rule instance to an instance of the mapped premises and conclusion, it induces a multifunctor* $\mathsf{Free}(S) \to \mathsf{Free}(S)$*, and derivability is closed under the map. Actions compose coherently: if two actions compose on sorts and on instances to a third, acting by the two in turn agrees with acting by the third on every derivation, up to the identification of sorts. In particular substitution of formulas for atoms acts on the intuitionistic and classical Hilbert multicategories, coherently with composition of substitutions, so Hilbert derivability is substitution closed as a consequence of freeness.*

Lean. `RuleSignature.InstanceAction`, `RuleSignature.actFunctor`, `RuleSignature.Derivable.act`, `RuleSignature.act_comp_heq`, `Hilbert.substFunctor`, `Hilbert.substFunctor_comp_heq`, `Hilbert.source_subst`

Remark 5.8 (Contexts, binders and side conditions). Object-level contexts and binders are part of the sorts: a HOL Light theorem carries its hypothesis set and its terms their binders, and side conditions such as freshness for abstraction are part of the rule-instance predicate. They change the generators, not the kind of structure (Section 6). Rules with one conclusion give a multicategory; multi-product consuming steps are treated in Section 9.

6 Kernels as free multicategories

A kernel with a decidable rule-instance interface $(\mathrm{W}, \mathrm{isInstance})$ presents the signature whose rule instances are accepted witnesses.

Theorem 6.1 (Derivability)**.** *Kernel derivability coincides with closed derivability in the free multicategory on the kernel's accepted witnesses.*

Lean. `RuleWitness.signature`, `RuleWitness.derivable_iff_derives`

Theorem 6.2 (Certificate trees are closed morphisms)**.** *For every conclusion b, closed derivations of b correspond bijectively to certificate trees of the kernel that its tree checker accepts with conclusion b. Erasure to trees is injective.*

Lean. `RuleWitness.closedEquiv`, `RuleWitness.toDerivation_injective`

Theorem 6.3 (Graphs unfold)**.** *Replaying a shared derivation graph with the unfolding step yields exactly the judgments of ordinary replay, now with closed derivations. Every node of an accepted graph denotes a closed morphism of its conclusion, and therefore an accepted certificate tree.*

Lean. `RuleWitness.termStep`, `RuleWitness.unfold_judgments`, `RuleWitness.unfold_root`, `RuleWitness.unfold_tree`

Theorem 6.4 (Graphs present exactly the closed morphisms)**.** *Every closed derivation is the unfolding of the last node of some accepted derivation graph. With the previous theorem, accepted graphs present exactly the closed morphisms of the free multicategory: sharing is a property of the representation, never of the morphism.*

Lean. `RuleWitness.presented`, `RuleWitness.presented_block`, `DerivationGraph.replayWith_relocate`, `DerivationGraph.replayWith_append`

Theorem 6.5 (Authored calculi)**.** *For every validated authored calculus, the existing open derivations with ordered premise contexts are exactly the terms of the free multicategory on its rule applications, and plugging derivation vectors is substitution. Every interpretation of the rule applications in a cartesian multicategory extends uniquely to open derivations, preserving substitution. Accepted raw open proofs decode computably to typed open derivations.*

Lean. `CertificateGSLT.ruleSignature`, `OpenDerivation.termEquiv`, `OpenDerivation.toTerm_bind`, `interpret_bind`, `interpret_unique`, `decodeOpen`

Corollary 6.6. *The classifying category of every authored calculus (VibeITP, projected Metamath snapshots, HOL Light's first-order calculus) is the category of contexts of a free cartesian multicategory, with products by concatenation.*

Lean. `ClassifyingContext.instCategory`, `ClassifyingContext.concatIsLimit`, `OpenDerivation.termEquiv`

Kernel	Rule-instance interface	Free multicategory
Intuitionistic / classical Hilbert	finite codes, profile-gated excluded middle	`RuleWitness.signature(interfacep)`
Extended Hilbert	classical codes and admitted definitions	`RuleWitness.signature(interfacehs)`
HOL Light	ten rules and policy axioms	`RuleWitness.signature(ruleWitnesspolicy)`
LRAT, RUP fragment	input clauses and RUP instances	`RuleWitness.signature(interfaceF)`
Authored calculi	rule applications	`CertificateGSLT.ruleSignature`
Grounded circular truth	ten codes over signed specifications	`RuleWitness.signature(Checking.interfacesystem)`
Guarded atom rules, presentations	original rule codes	`Presentation.ruleWitness`

7 Translations as multifunctors

By Theorem 5.4, a translation is determined by the images of rule instances; derivability is preserved automatically.

Theorem 7.1 (Faithful renamings)**.** *A sort-preserving renaming of rule instances that is injective induces an injective multifunctor.*

Lean. `RuleSignature.rename`, `RuleSignature.lift_rename_injective`

Theorem 7.2 (Intuitionistic into classical)**.** *The inclusion of intuitionistic into classical Hilbert rule instances induces a faithful multifunctor that preserves but does not reflect derivability: excluded middle is classically derivable and not intuitionistically derivable.*

Lean. `Hilbert.inclusion`, `Hilbert.inclusion_faithful`, `Hilbert.inclusion_not_reflecting`

Theorem 7.3 (Classical into extended)**.** *For every list of definitions, the inclusion of classical into extended rule instances induces a faithful multifunctor. For admitted fresh definitions it is conservative on claims in the old vocabulary. It is not full: citing a definition is a derivation in the image of no classical derivation.*

Lean. `Extension.inclusion`, `Extension.inclusion_faithful`, `Extension.inclusion_conservative`, `Extension.inclusion_not_full`

Theorem 7.4 (HOL Light into its first-order calculus)**.** *Each accepted HOL Light rule instance is translated by the node step on its canonical premise context and decoded to a typed open derivation of its rendered conclusion. This determines a unique multifunctor from HOL Light's free rule multicategory to the free multicategory of its first-order calculus over the rendering of theorems. It preserves derivability, and for admitted axiom lists reflects it: the calculus derives the rendering of a sequent exactly when HOL Light derives a theorem with that sequent. Positive control:* $\vdash p = p$ *by REFL; negative control:* $\vdash \mathsf{F}$*.*

Lean. `HOLLight.FirstOrder.generatorImage`, `translation`, `translation_unique`, `translation_derivable`, `translation_reflects`, `Kernel.reflexivity_translated`, `Kernel.falsity_not_translated`

Theorem 7.5 (Sharing commutes with translation)**.** *For any interpretation of a kernel's rule instances in a free multicategory, translate a shared derivation graph node by node: check each node against its source premise sorts and instantiate its rule image with the translated premises. From translated entries this computes exactly the multifunctor images of the unfolded entries. Hence every accepted graph translates with one rule image per node, and the value at each node is the multifunctor image of that node's closed derivation. In particular this holds for HOL Light graphs and the translation above.*

Lean. `RuleWitness.liftStep`, `RuleWitness.liftStep_liftEntry`, `RuleWitness.replay_liftStep`, `RuleWitness.shared_translation`, `HOLLight.FirstOrder.shared_translation`

Remark 7.6. The existing compilation of first-order calculi into MM0 (the system of *Metamath Zero: The Cartesian Theorem Prover*[8]) and the existing proof-graph translation with shared lemmas complete the chain to MM0 at the level of certificates. That existing translation builds each lemma with the same node step over the cumulative context. On all 25 nodes of the three real HOL Light exports of Section 12, each of its lemmas coincides with the raw rule proof from which the multifunctor's rule image is decoded, after renaming premise positions to the lemma's parents. This is an executed comparison; the general equality is open (Section 15).

8 Costs: logical work, sharing and extension

Theorem 8.1 (Additivity)**.** *Rule nodes are additive under substitution, each hole occurrence charged once:* $\mathsf{size}(t[\sigma]) = \mathsf{size}(t) + \sum_{i \in \mathsf{occ}(t)} \mathsf{size}(\sigma_i)$*.*

Lean. `Term.size_subst`

Theorem 8.2 (Affine translations are linear on trees)**.** *If every rule image uses each premise at most once and has at most* c *rule nodes, the induced multifunctor satisfies* $\mathsf{size}(Ft) \le c \cdot \mathsf{size}(t)$*.*

Lean. `RuleSignature.lift_size_le`

Theorem 8.3 (SHARE: a contraction-induced expansion gap)**.** *Translate the chain calculus (one premise per step) into the duplication calculus by an image citing its premise twice. The* k*-chain has* $k+1$ *nodes; its translation is a tree with* $2^{k+1}-1$ *nodes; the duplication calculus has an accepted shared graph with* $k+1$ *rows for the same conclusion. The contracting image is not affine.*

Lean. `Contraction.contraction_gap`, `Contraction.image_not_affine`, `ProofSharingBounds.no_tree_sizeSimulation_dag`

Remark 8.4 (EXTEND). Extension by definitions changes the multicategory: the inclusion is faithful and conservative but not full (Section 7), and literal expansion of an admitted doubling chain has $2^{k+1}-1$ formula nodes from encoded size at most $5(k+1)(k+2)$ (`PropositionalExtension.literal_expansion_gap`). Sharing never changes the multicategory: a graph and its unfolded

tree are one morphism. In the sense of Cook and Reckhow, *The Relative Efficiency of Propositional Proof Systems*, a translation with polynomial size and time bounds is a p-simulation. The proved size separation rules out polynomial-size translation from the shared duplication certificates to trees; the straightforward tree-to-DAG representation comparison has no such size obstruction. An executed compiler-time bound must be supplied separately. Extended resolution's polynomial simulation of DRAT (Kiesl, Rebola-Pardo and Heule, *Extended Resolution Simulates DRAT*) is the corresponding EXTEND phenomenon for SAT certificates.

Proposition 8.5 (Time accounts)**.** *The p-simulation of HOL Light proof graphs by MM0 has a degree-ten certificate bound and a degree-ten declared time account; the declared time is at least the weight of the lemmas the translation writes. An operational simulation derives its clock from a finite-stack machine execution, and such clocks are positive.*

Lean. `HOLLight.FirstOrder.graphTranslate_weight`, `graphTime_ge_weight`, `Kernel.mm0Simulation`, `OperationalPSimulation.toPSimulation`, `OperationalPSimulation.time_positive`

Status. The HOL Light to MM0 time account is declared, bounded below by output weight; its finite-stack realization is open.

9 Atomic multi-pattern steps and schedules

A step keeps some occurrences, consumes others and produces atoms, when its rule admits them; this is the shape of multi-headed CHR simpagation (Di Giusto, Gabbrielli and Meo, *On the expressive power of multiple heads in CHR*) and of chemical-style rewriting (Berry and Boudol, *The Chemical Abstract Machine*).

Theorem 9.1 (Local characterization of replay)**.** *Sequential replay with any local step accepts exactly when the final cache extends the initial one, has one value per node, every reference points backward, and every node's step on its looked-up premises yields its value.*

Lean. `DerivationGraph.replayWith`, `DerivationGraph.replayWith_iff_local`, `DerivationGraph.replayWith_judgmentStep`

Theorem 9.2 (Schedule independence)**.** *For a step that ignores physical reference positions, reordering a graph by any permutation that keeps premises before consumers, with references relocated, accepts exactly when the original order does (for topological originals), and moves each node's value to its new position. A reordering that places a premise after its consumer is refused. The numbers of local steps and premise lookups are invariant. In particular judgment replay and unfolding are schedule independent.*

Lean. `DerivationGraph.replayWith_permute`, `replayWith_permute_reflect`, `replayWith_permute_iff`, `replayWith_permute_refused`, `permute_lookups`, `replay_permute`, `RuleWitness.unfold_permute`

Theorem 9.3 (Monotone steps are derivations)**.** *Without consumption, running single-product steps is replay of the corresponding derivation graph, and every accepted trace denotes, at every step, a closed morphism of the free multicategory on the step rules. A monotone step with several products runs exactly as the sequence of its single-product components.*

Lean. `MultiPremiseSteps.run_monotone`, `run_monotone_unfolds`, `run_split`

Theorem 9.4 (Consumption)**.** *In an accepted trace a consumed occurrence is never cited again; occurrences may be kept twice but not consumed twice, nor kept and consumed together. Two steps competing for one occurrence lead to different stores, neither of which can fire the other. Two steps with independent footprints (neither consumes what the other cites, and the second cites no product*

of the first) fire in either order, with the same live atoms up to the order of their products. In the formalized tree of single-wire cuts, a composite has one internal wire fewer than generators. The split/join process has two steps and two internal wires, so that compositional syntax cannot represent it while preserving those counts. This rules out that syntax as a complete account; it does not rule out encoding processes in categories or alternative polycategorical frameworks.

Lean. `MultiPremiseSteps.consumed_not_cited`, `Controls.consumption_not_confluent`, `Controls.keep_twice`, `Controls.take_twice_refused`, `independent_commute`, `Controls.independent_steps_commute`, `PolyComposite.wires_add_one`, `process_not_polycategorical`

Remark 9.5 (What the correspondence keeps). Kept: rule witnesses, ordered and repeated premise occurrences, every conclusion, acceptance including refusal of unused bad steps, and step counts. Forgotten: the order of independent steps; sharing (a graph and its unfolding are one morphism); the identity of steps not used by the selected root. Consumption keeps resources linear and needs monoidal composition of processes; negative reads are outside this step shape.

10 Admission and revalidation of retained evidence

Theorem 10.1 (Exact revalidation)**.** *For a graph accepted under one interface, rechecking each node's rule instance against another interface, with premises read from the retained cache, succeeds exactly when the mapped graph replays under the new interface to the same cache. Admitted extensions (witness maps keeping accepted instances accepted) transport evidence; checked judgments are reusable in larger rule systems; a retained source-address schedule replays to the identical state under a new interface accepting each instance it used.*

Lean. `DerivationGraph.revalidate_iff`, `replay_extend`, `CheckedJudgment.extend`, `BorrowedCursor.run_reinterpret`

Theorem 10.2 (Propositional admission joined to retained evidence)**.** *Every accepted intuitionistic graph is accepted classically with the same cache. An accepted extended graph remains accepted when further admitted definitions are appended. Classical evidence for excluded middle is refused on intuitionistic revalidation; evidence citing a definition that was replaced by a different admitted definition is refused.*

Lean. `Revalidation.intuitionistic_to_classical`, `Revalidation.checkDAG_append`, `Controls.em_refused_intuitionistically`, `Controls.worked_extended`, `Controls.worked_changed_refused`

11 Module coverage and the LRAT action adapter

Theorem 11.1 (HOL Light kernel module)**.** *The first-order calculus of HOL Light over an axiom list renders as a portable module that the verified reader and the original validator load exactly. The loaded package derives the rendering of a sequent exactly when HOL Light derives a theorem with that sequent, for admitted axiom lists.*

Lean. `HOLLight.FirstOrder.kernelModule`, `kernelModule_read`, `kernelModule_faithful`

Status. Exported for the bare kernel: 147 rules, 991 249 bytes, read back through the actual reader and validator.

Theorem 11.2 (LRAT actions)**.** *A reverse-unit-propagation addition is a multi-premise rule instance whose premises are its hinted clauses, and it is sound. An LRAT action stream compiles, with identifiers resolved to positions and deletions enforced, to a derivation graph; acceptance of that*

graph with an empty clause refutes the formula, and every action denotes a closed morphism of the free multicategory on RUP instances. A resolution-asymmetric addition need not be implied, so it is not a rule instance of this calculus.

Lean. `PropositionalLRATActions.rupCheck_sound`, `check_sound`, `check_graph`, `check_unfolds`, `rat_not_implied`, `rat_not_rule_instance`

Native profile	Faithful presentation	Portable module
VibeITP	authored calculus, `kernel_derives_iff_classifyingArrow`	yes, 123 schemas
Metamath snapshot	projected authored calculus	yes, 25 schemas
HOL Light kernel	first-order calculus, `kernelModule_faithful`	yes, 147 rules
Hilbert, extended Hilbert	exact rule codes; sorted Hilbert source presentation also exists	not qualified here
LRAT (RUP fragment)	action calculus, `check_sound`	no (RUP is a computed check)
Grounded circular truth	signed evidence rules, `judgment_coverage`	no (base admission is a parameter)
MM0, Megalodon	owned by the active MM0 lane	not covered here
Rho, MeTTa equations, GSLT recipes	retained workers (*Abstract Machines*, Section 13)	open

12 The costed HOL Light to MM0 translation

The existing receiver replays real HOL Light exports, lowers the registered receipts to proof graphs, translates each node through its first-order step, compiles the shared lemmas to MM0 saved proofs, checks them in Lean and prints specification and proof texts. The reference MM0 tools then compile the proof text and verify the binary against the specification.

Export	events	spec (MB)	MMU (MB)	emit (s)	MMB (bytes)	compile (s)	verify (s)
T_DEF	1	26.2	40.1	14.4	220 512	0.58	0.13
TRUTH	8	22.6	252.2	2166	223 536	4.08	0.14

Emission includes the Lean-side check and printing of each text: for TRUTH, lowering, node translation and shared-proof compilation take under a minute, while checking and printing the proof text and the MM1 text take 16 and 19 minutes at a peak of 2.8 GiB. The specification encodes the fixed 147-rule presentation and dominates the binary certificate; the eight-node proof adds about three kilobytes to it. The proved bounds are polynomial of degree ten with generous constants; the measured growth is far below them. The text formats and the in-process checking dominate wall-clock cost, not the reference verifier. These are executed times of the existing binary and are kept separate from the logical work and the declared time account of Section 8, which the following table reports in the measures of the proved bounds (graph size N, lemma-certificate weight, first-order rule nodes and declared time).

Export	nodes	N	certificate weight	rule nodes	declared time
T_DEF	1	371 826	185 913	1	1.9×10^{22}
TRUTH	8	3 610 421	4 562 433	1 505	1.3×10^{24}
core bool	16	43 070 295	24 292 374	1 513	5.2×10^{28}

These measurements are historical qualification runs of the receiver frozen on 13 October 2026. They have not been reproduced with a rebuilt receiver in the present source state and are not current performance guarantees.

The certificate weight is $1.26\,N$ on TRUTH, while the proved bound $(35 \cdot 10^{12} + 2)(N+1)^{10}$ exceeds 10^{78}: the degree-ten bound is sound and very loose. The declared time is dominated by its fourth-power comparison term and says nothing about executed time. On the sixteen-event core bool export, the executed compilation to MM0 saved proofs exhausted its available memory at the last of its sixteen nodes, after assembling a 156-rule presentation; memory, not time, bounds the executed translation there.

13 Dependent judgments, theorem authority and set models

Four independent questions determine a kernel interface. Keeping them separate permits the same computation to support several faithful consumers.

Coordinate	Declared contract
Judgment	What follows, in which context and theory: typing, equality, reachability, one answer, or a complete result observation.
Evidence	What is checked: local rule data, a tree, a shared graph, or a finite trace of licensed transitions.
Authority	How checked conclusions can be constructed and used; which profile and declaration snapshot authorize them.
Observation	What execution retains or exposes: verdict, pending work, cost, dependencies, or an optional exportable history.

An LCF interface protects theorem construction. It can also record proof terms or a journal. Certificate checking can mint the same logical kind of value without retaining the certificate. Neither choice supplies a constant checking cost, proves a native seal implementation, or recovers evidence that execution erased. A checked finite trace can be evidence when its transition rules establish the intended conclusion. A trace of an unrelated implementation is not automatically a derivation of that judgment.

The MeTTa equation kernel is sound and complete for its native reachability judgment. Full HE or PeTTa evaluation, resource consumption and mathematical multi-premise inference require their respective judgments. No single one of these observations defines every useful MeTTa kernel. The design of a MeTTa kernel integrating these judgments remains open; the following connection uses an existing mathematical calculus.

13.1 Finite dependent evidence through the common interfaces

The selected profile is the existing cumulative tower over natural-number level expressions. Its syntax has dependent functions and pairs, identity formation and reflexivity, explicit universe heads, and cumulativity. Its formation-sensitive typing relation has fourteen rules. Constants and root computations are parameterized in the general construction; this particular tower declares neither. Structural beta computation remains available. Only this declared mathematical profile is instantiated here.

Local codes contain syntax and rule selections. In the conversion rule they contain an existing finite structural conversion code. Decoding checks the local universe, declaration and head conditions and checks that conversion code; it does not receive a proof of conversion as executable input. The decoded node is an instance of the independently defined rule signature. The common tree and DAG replay algorithms therefore preserve the original premise positions and dependent substitutions. The maintained structural replay checker also publishes common checked judgments directly, and its existing telescope certificates check the ambient context.

Theorem 13.1 (Exact finite authority for the selected judgment)**.** *For the selected tower, a formation-sensitive judgment in a formed telescope holds if and only if there is an accepted finite request. Such a request checks its context, every graph row, the root address and its actual*

conclusion. Raw typing derivability is equivalently accepted tree evidence, accepted graph evidence, or a minted common checked judgment.

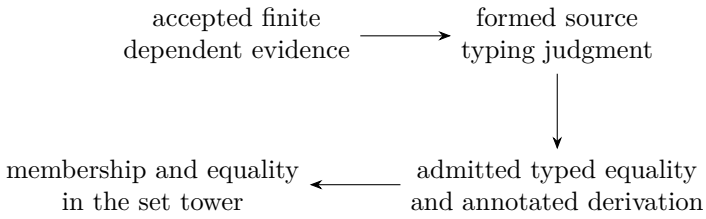
Lean. `FormationSensitiveChecking.typing_iff_checked_tree`, `FormationSensitiveChecking.typing_iff_checked_graph`, `FormationSensitiveChecking.typing_iff_minted`, `FormationSensitiveChecking.mintReplay_iff`, `FormationSensitiveChecking.judgment_iff_checked`, `Tower.Checking.coverage`

Exactness is existence of accepted evidence, not a terminating procedure for finding it. This is a connection of scoped Lean data and checked algorithms, not a portable serialization theorem or a refinement of CeTTa's C implementation. The source formation-sensitive conversion is structural conversion; the comparison into the stronger typed equality does not make every typed eta identification a source conversion.

A raw variable rule can hold in an unformed telescope. Such a raw certificate must not authorize a formed request. The controls also reject changed root claims, missing roots, forward or cyclic dependencies, invalid universe lifts, and invalid unused suffixes. Shared references retain all premise occurrences; they do not duplicate consumable execution authority.

13.2 The comparison to sets

The diagram keeps the source judgment between evidence and meaning:



Theorem 13.2 (Checked evidence has its declared set meaning)**.** *Under the explicit hypothesis* `CofinalInaccessibles`*, every accepted request yields an annotated derivation whose telescope, subject and displayed type erase to that request and whose statement holds in the existing set tower. Checked subject values belong to their displayed type values. Finite checked conversion between independently admitted endpoints preserves their actual model values.*

Lean. `TowerInterpretation.Checking.request_model_sound`, `TowerInterpretation.Checking.checked_value_member`, `TowerInterpretation.Checking.checked_type_conversion`, `TowerInterpretation.Checking.checked_term_conversion`

For closed universe-membership queries the comparison also reflects the model: a finite request for $U_c : U_d$ exists exactly when the corresponding set universe is a member of the other. The same equivalence holds after the existing well-founded embedding into material hypersets. This is a scoped reflection result for universes, not completeness for arbitrary set truth. Under the named model hypothesis, every request for a closed inhabitant of the existing empty type is rejected.

Lean. `TowerInterpretation.Checking.universe_request_iff_set`, `TowerInterpretation.Checking.universe_request_iff_hyperset`, `TowerInterpretation.Checking.empty_request_refuses`

Substitution uses the actual checked component terms. Their accepted requests construct the source contextual arrow. At the semantic comparison boundary, the annotated components carry their typing and erasure witnesses; evaluation of those components constructs the satisfying-environment map. The value of a substituted checked term equals its original value in that substituted environment. Context and component compatibility are retained in the statement, rather than being replaced by an assumed semantic action.

Lean. `FormationSensitiveChecking.checked_substitute`,
`TowerInterpretation.Checking.checkedArrow`,
`TowerInterpretation.Checking.checked_value_substitution`

The model map has several distinct scopes. The cumulative tower has the set interpretation just used, with its named inaccessible-cardinal hypothesis. The HOTG-style mathematical packages add declared set operations and theory assumptions; they are not automatically translations to the native Megalodon Mathdata checker. The material hyperset family construction takes an explicit presentation and supplies contexts, dependent families, substitution and Pi/Sigma structure. Its extensional identity does not establish a faithful interpretation of arbitrary higher paths, cubical operations or every dependent profile.

A useful negative boundary is the existing HOTG-style `setTheory_not_admitted`: a package can have a set model while failing its stated computational admission discipline. The rule-constant alternative has separate preservation results. Semantic equality therefore cannot be promoted to checker conversion merely because both expressions denote the same set. Conversely, a model interpretation need not reflect all set truth back into source derivations. These comparisons preserve their assumptions; they assert neither equivalence of all foundations nor optimal native speed.

13.3 Operational logical form and relative checking bases

The choice of a low-tier checker can follow from a richer dependent theory and the fragment requested of it. There are at least three constructions: restrict declarations and computation rules; elaborate implicit conversion into explicit evidence; or specialize a checking algorithm for a declared fragment. These give different proof formats and cost profiles. A literal syntax intersection does not decide which construction is best.

Dependent carve-outs additionally retain formed contexts, binding, substitution, universe policies and the selected identity rules. Their operational predicate comparison does not discharge these dependent laws or admit an inductive schema. A fibration organizes the obligations; an executable extraction and an optimality theorem require further evidence.

For the existing dependent calculus, every accepted formed request has a finite sufficient restriction of its declaration and root-computation package. The same context, subject and displayed type are derivable there, and every judgment of that restriction reflects to the original package. This is a constructed restriction, not a theorem that a smallest checker exists. The universe policy is retained. Computational admission and an executable root decoder for the restriction are additional obligations: pruning may remove formation or preservation support required by a chosen admission discipline.

Lean. `FormationSensitiveCheckingOSLF.checked_request_has_restriction`,
`FormationSensitive.Dependencies.exists_restriction_with_reflection`,
`FormationSensitive.RestrictionPreservation.oslf_observation_does_not_admit_restriction`,
`FormationSensitive.RestrictionPreservation.oslf_observation_with_preserving_repair`

OSLF applies to the declared dynamics. In the conversion-evidence observation, vertices are the original scoped terms and edges are original structural steps with either orientation. The observation theory retains syntax equality as its equation policy, so conversion remains visible as finite paths. Its generated reachability native type holds exactly when the independently authored conversion holds, equivalently when some supplied finite code checks. Reverse beta is therefore a valid conversion edge; this does not declare it an evaluator reduction. A raw beta path also does not establish formed typing.

Lean. `FormationSensitiveCheckingOSLF.path_iff_conversion`,
`FormationSensitiveCheckingOSLF.satisfies_iff_checked`,
`Tower.Checking.Controls.reverse_beta_oslf`, `Tower.Checking.Controls.raw_beta_oslf`

Native type theory obtains richer structure through the internal languages of presheaf toposes [6]. The comparison here concerns the declared relational OSLF predicate frame and its operational

routes. It does not establish that entire presheaf construction for every dependent profile.

In the general OSLF construction, semantic predicates are invariant under the selected equations. Closure under computation is a further property. The existing route interfaces yield the following separate results:

Route contract	Earned observation law
Equation-respecting map	Reindex invariant predicates. Pointwise agreement modulo target equations gives equal predicate actions.
Strict forward operational translation	Preserve existential one-step observations; additional target behavior may remain.
Locally covered translation	Also reflect those observations and finite reachability from image states.
Path-valued operational realization	Preserve finite reachability while permitting a source step to become zero or several target steps.

Lean. `EquationRespectingMap.pullback_eq_of_equiv`, `OperationalTranslation.preserves_semanticDiamond`, `CoveredTranslation.semanticDiamond_iff`, `CoveredTranslation.semanticReach_iff`, `OperationalRealization.preserves_semanticReach`

The extra-target-step control refutes unrestricted reflection. The fusion control preserves a path observation while destroying a one-step diamond. OSLF's right-adjoint box in this construction observes incoming steps; its transport cannot be inferred from outgoing coverage alone. Bounded-cost observations additionally require a declared charge and a bound on mapped paths. Predicate transport does not by itself establish those bounds, proof translation, native seal enforcement, or equality of retained proof histories. The basic behavioral GSLT morphism only promises bisimilarity preservation.

Lean. `IndexedOperational.Canary.escaping_native_type_not_reflected`, `IndexedOperational.Canary.outgoing_coverage_does_not_preserve_past_box`, `OperationalRealization.FusionCanary.fusion_changes_one_step_diamond`

HOL Light, MM0 and Megalodon consequently remain comparison points and native realizations with explicit source contracts. They do not exhaust the possible checking bases for dependent profiles. A semantic set model is another comparison edge, with its assumptions retained; it is not an executable proof translation or a reflection theorem for arbitrary set truth. Relative optimality needs the fragment, certificate representation, operations and cost model to be fixed before it can be asserted.

For the selected cumulative profile, the guarded set-model comparison also receives this OSLF conversion observation directly. Two accepted term-typing requests in a formed telescope give equal actual set values when the conversion-native observation holds. The annotated context and the named model hypothesis remain explicit.

Lean. `TowerInterpretation.Checking.oslf_term_conversion`

13.4 A computational STT carve-out, with a proved boundary

The existing one-base simply typed lambda calculus supplies a first exact comparison. Write $S(\Gamma, A)$ for its intrinsically typed terms, E for its embedding in the root-free cumulative Pi/Sigma/identity tower, and H_b for its HOL interpretation at a chosen base sort b. Simple arrows become nondependent products in E and ordinary arrows in H_b. The chosen base sort remains distinct from HOL's proposition sort. For every context and simple type:

$$\begin{aligned} \mathrm{Join}_\beta(H_b(s), H_b(t)) &\iff s \equiv_\beta t \\ &\iff \mathrm{Conv}_{\mathrm{struct}}(E(s), E(t)) \\ &\iff \mathrm{decideConversion}(s,t) = \mathsf{true}. \end{aligned}$$

The tower conversion path may leave the simple image. Reflection follows from normalization and confluence, not an assumption that all intermediate terms remain simple. HOL uses its independently defined operational beta steps. Normal intrinsic terms remain normal there, so common reducts determine the same intrinsic normal form.

Lean. `SingleBaseSTLC.HOLComparison.holJoinable_iff_betaConv`,
`SingleBaseSTLC.HOLComparison.holJoinable_iff_towerConv`,
`SingleBaseSTLC.HOLComparison.decideConversion_iff_holJoinable`

The scope is one base sort, variables, abstractions and applications, with no declared constants. This compares a computational judgment, not all HOL theoremhood or arbitrary dependent inhabitants. It is a reference component for a possible native kernel, not a completed HOL bubble or a selection among the dependent language candidates.

The maps commute with simultaneous substitution and give a functor between the existing context categories. An indexed family alone is not a fibration: chosen cartesian lifts and their laws require additional constructions. Typed HOL syntax retains intermediate application types that the unannotated tower erasure can lose, even at a fixed result type. Conversion classes remain faithfully compared. Raw syntax recovery and conversion reflection are therefore different requirements on a shared representation.

Lean. `SingleBaseSTLC.HOLComparison.contextFunctor`,
`SingleBaseSTLC.HOLComparison.substitute_term`,
`SingleBaseSTLC.HOLComparison.Controls.hol_retains_erased_types`,
`SingleBaseSTLC.ConversionDecision.betaClass_toTower_injective`

Two local policies. The common bubble interface now has a simple-term instance. Beta steps retain HOL operational evidence; a second policy additionally permits eta, with application and abstraction congruence in both policies. Free conversion trees retain their composition and intermediate states. Each tree elaborates constructively to an ordinary retained HOL equality proof with the same endpoints. The beta-only policy presents exactly the structural tower conversion above.

Lean. `SingleBaseSTLC.HOLConversionBubble.elaborate`,
`SingleBaseSTLC.HOLConversionBubble.beta_path_iff_tower`

For an arrow-typed variable f, the terms f and $\lambda x.fx$ are beta-normal and not beta-convertible, although the eta policy relates them and HOL proves their equality. The tower's *typed-equality* judgment also contains eta; its *structural conversion* in the comparison does not. Even two judgments over one package cannot be interchanged silently. An equality theorem does not automatically license definitional computation. Arbitrary additional rewrite packages would need fresh admission obligations.

Lean. `SingleBaseSTLC.HOLConversionBubble.Controls.eta_strictly_adds_conversion`,
`SingleBaseSTLC.HOLComparison.Controls.hol_equality_does_not_reflect_conversion`,
`SingleBaseSTLC.HOLTowerModel.typed_equality_not_structural_conversion`

Operational predicates, dependent readings and models. This conversion observation and the earlier dependent checking development reuse one general construction for the equivalence closure of an authored relation. Equations of the observation theory are syntax equality; bidirectional computation edges remain visible. Its OSLF reachability predicate has exactly the support of retained conversion trees. On the beta fragment the existing total intrinsic normal-form comparison decides it. Predicate truth does not reconstruct a particular history.

Lean. `IndexedOperational.conversionTheory_path_iff`,
`IndexedOperational.satisfies_conversionNativeType_iff`,
`SingleBaseSTLC.HOLConversionBubble.satisfies_iff_path`,
`SingleBaseSTLC.HOLConversionBubble.beta_satisfies_iff_decides`

Every lawful dependent `HOLReading` reads these pure simple terms without interpreting additional

constants or supplying an equality-proof algebra. Their computed codes are typed in that reading's actual selected judgment. The maintained native substitution constructor has no unread image on this fragment, and code commutes with substitution. The beta observation thus gives typed equality of the computed codes. This is a parametric inclusion result with explicit reading laws, not reflection from arbitrary dependent proofs.

Lean. `SingleBaseSTLC.HOLReadingComparison.code_typed`,
`SingleBaseSTLC.HOLReadingComparison.code_substitute`,
`SingleBaseSTLC.HOLReadingComparison.observed_typed`

For the tower, the independently formed simple context and admitted equality reach its maintained annotated set interpretation under the named `CofinalInaccessibles` hypothesis. That hypothesis belongs to the semantic edge, not the beta decision procedure. The earlier well-founded embedding into material hypersets retains its stated scope; neither result supplies arbitrary higher identities or a translation to native Mathdata.

Lean. `SingleBaseSTLC.HOLTowerModel.observed_set_equality`

From a component to a whole HOL bubble. Choice, infinity and classical principles belong to a declared theory and its interpretations; conversion policy belongs to the judgment. Option names are not independence proofs for their combinations. The existing HOL calculus, retained derivations and theory interfaces supply these logical profiles. Merely naming another kernel does not require copying their rules.

A complete proof-level carve-out needs a rule-preserving interpretation into the selected dependent judgment and a genuine reflection or conservativity argument for the declared image. Type-shape membership supplies neither. Routes to native HOL Light, Megalodon and configured MM0 must retain their own assumptions, definitions, equality policies and evidence. This comparison establishes no new complete three-way translation.

A suitable tier-zero realization can specialize a richer checker, or expose its implicit computation as explicit evidence. It need not be a literal subset of the checker's source or add an interpreter layer to a qualified native operation. Direct normalization, finite replay, optional proof recording and protected theorem authority have separate contracts. Choosing their best physical realization requires workload and cost comparisons.

13.5 Conversion policies, proof elaboration and operational fibres

A checking graph needs more than an ordering of mathematical strength. One edge may add an axiom, another may license a formerly explicit equality as conversion, and another may change only the producer of certificates. An interpretation, a conservative extension and a measured specialization carry different laws. Distinct proof and execution histories remain visible until an observation explicitly forgets them.

The current HOL reference instance fixes arbitrary base sorts and typed constants, while a policy selects beta permissions by the contracted redex's result type, eta permissions by domain and codomain, and equation permissions by declaration occurrence. The permission is checked at the actual inner redex, even when the surrounding term has another type. A cited equation carries its pattern telescope and a simultaneous substitution. Explicit typed substitution contexts and abstraction/quantifier congruence support computation under binders without dropping scope. A substitution context can use its parameter repeatedly; this derived equality action is not necessarily one evaluator contraction.

For a policy P, let R_P be the independently stated local-step relation. The maintained equality-recipe checker computes endpoints, checks every transitivity join and has exact authority:

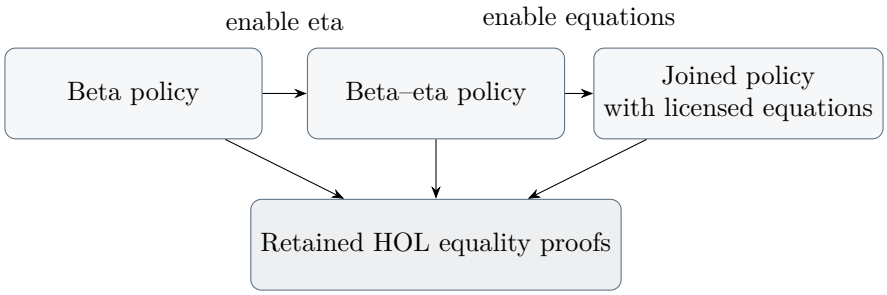
$$s\ \mathrm{EqvGen}(R_P)\ t \quad \Longleftrightarrow \quad \exists e.\ \mathrm{check}_P((s,t),e) = \mathsf{true}.$$

This is a decision about a supplied finite article. It does not require a normalizer or decide whether some article exists. The generic decoder constructs a free conversion tree from successful replay; forgetting that tree recovers the submitted recipe exactly, including its local witnesses and binary composition structure.

Lean. `EqualityRecipe.checkedPathWithRecovery`, `EqualityRecipe.forget_checkedPath`, `HOL.ConversionRecipes.leaf_authority`, `HOL.ConversionRecipes.authority`

The existing typed beta-execution carrier also compiles directly into these recipes, retaining each contracted position. A previous beta-joinability witness consequently supplies an accepted beta-only article. This is a computed forward connection to the maintained operational component; it does not assert reverse reflection for every policy or equality of proof histories.

Lean. `HOL.ConversionRecipes.executionRecipe`, `HOL.ConversionRecipes.executionRecipe_checked`, `HOL.ConversionRecipes.joinable_has_betaRecipe`



The downward edges use actual retained equality receipts for each equation, at its original telescope. They construct ordinary HOL proofs through beta, eta, congruence and equality composition. Logical retyping becomes explicit propositional equality elimination. The two-sort controls reach the same endpoint equality by primitive eta and by an instantiated eta declaration, while the submitted receipts remain distinct. Another pair has the same endpoint equality and retained proof sizes one and three. These are proof sizes, not elapsed-time comparisons.

Lean. `HOL.EquationProofs.instantiate`, `HOL.ConversionRecipes.elaborate`, `HOL.ConversionRecipes.convertProof`, `HOL.ConversionRecipes.Controls.eta_and_equation_recipes_distinct`, `HOL.ConversionRecipes.Controls.proof_costs_differ`

The existing implication/universal modulo calculus stores proposition-valued conversion. With the same equation receipts, every one of its proofs has an ordinary retained HOL proof at the original ordered sequent. This translation establishes existence; it does not reconstruct a forgotten source history. The checked recipe interface supplies that stronger reconstruction. Curry's cyclic equation cannot acquire the required HOL receipts: otherwise its existing modulo proof of $\forall p.p$ would become an ordinary HOL proof, contradicting the maintained standard model. These edges cover equations already justified by the target calculus. Fresh definitional extensions instead use the separate signature-changing unfolding and conservativity construction.

Lean. `HOL.ProofSyntaxModulo.hasHOLProof`, `HOL.ConversionElaborationControls.no_curry_equationProofs`, `HOL.ProofSyntaxModulo.definition_conservative`

What composes. Two rule packages independently admissible over the *same original* judgment system can be combined without changing derivability. Actual local certificate compilers also combine, selecting each feature's own expansion and node budget. The mixed control eliminates a shortcut and an identity inference into a three-node base certificate; adjoining the genuinely new fact still has no such compiler. This logical composition law does not establish normalization, confluence or decidability for a union of conversion engines. Those are separate interaction obligations.

Lean. `extendRules_union_sameDerivability_iff`, `AdditionalRuleCertificateCompiler.combine`, `AdditionalRuleCertificateCompiler.combine_sameDerivability`, `ConservativeExtensionCanary.compiledMixed_count`, `ConservativeExtensionCanary.combined_newFact_forbidden`

Elaboration and execution. An elaborator may perform implicit-argument search, instance resolution, unification or tactics. Here an arbitrary producer submits finite recipes to the existing screening interface, and only accepted submissions acquire retained equality proofs. Producer completeness is unnecessary for this soundness boundary. This is a concrete consumer of the producer/checker separation, not an implementation of all those frontend algorithms.

Lean. `HOL.ConversionRecipes.elaborateSubmission`, `HeuristicEvidence.arbitrary_producer_sound`

The operational fibre over a context, type and policy has the original typed terms as states. Its OSLF reachability predicate is exactly certificate existence for $\mathrm{EqvGen}(R_P)$. Policy extensions and joins preserve this observation, including computations under binders. This predicate forgets the chosen recipe; the recipe decoder does not. Program-execution fibres additionally retain the chosen world, resources, representation and scheduling policy. Their routes use the earlier strict, covered or path-valued operational contracts as appropriate. Logical conversion is not thereby declared to be evaluator reduction, and execution costs need their own accounts.

Lean. `HOL.ConversionRecipes.observed_iff_accepted`, `HOL.ConversionRecipes.LocalStep.conversion_mono`, `HOL.ConversionRecipes.observed_mono`

Extensional comparisons remain selected bridges. Modular type-theory presentations provide an established precedent for feature-selected fragments [11]. Hofmann's comparison of specified extensional and intensional theories, with functional extensionality and UIP on the intensional side, concerns an interpretation and conservativity, rather than identical conversion algorithms. The later Morita-equivalence result compares their categories of models in its stated setting [12, 13]. These results do not establish such a bridge for every dependent candidate in this library. A semantic extensional readout can coexist with an executable intensional checker and explicit transport evidence, without making equality reflection a rule of every checker. Chosen route action, universe policy, theory assumptions and operational observations remain part of the comparison.

The new checker is a proof-producing reference on already scoped and typed syntax. It qualifies neither a raw-byte parser nor native seal enforcement, and it supplies no native performance comparison. Full dependent-fragment reflection and the routes to actual HOL Light, Mathdata and configured MM0 remain further graph edges.

13.6 Selected iota roots and checked dependent instantiation

The existing `ParameterizedPiSigmaId` rules already state the head, universe, cumulativity, constant and root-computation policies of that calculus. Its indexed-family interface separately carries constructors, an eliminator specification and typed iota schemas. The following checking connection consumes those schemas; it does not derive another eliminator or introduce a universal profile record for every dependent candidate.

An ordered schema table T stores open endpoint pairs at their respective arities. A policy $e : \mathrm{Fin}\,|T| \to \mathsf{Bool}$ selects original row occurrences. A finite root code supplies a row address and a list of scoped arguments. Replay checks the address, permission and argument count, then computes capture-safe simultaneous substitution into the selected pair. The independently defined root evidence retains both the original row and the substitution. Its decoder is sound and complete for exactly those instances. Repeated equal rows remain distinct citations, and an enabled duplicate cannot rescue a disabled citation.

Lean. `SchemaTableConversionCode.retained`, `SchemaTableConversionCode.decode_sound`, `SchemaTableConversionCode.decode_complete`, `SchemaTableConversionCode.rootDecoder`

Renaming and substitution act on every submitted argument while retaining the row address. Replay commutes with these actions, including refusals. Selection by schema occurrence is stable under substitution. A selection depending on the instantiated dependent type would need its own stability law before becoming a root-computation policy.

Lean. `SchemaTableConversionCode.decode_rename`, `SchemaTableConversionCode.decode_substitute`

For a table of existing typed iota schemas, row lookup also retrieves the original telescope Γ_i, displayed type A_i and retained computation witness. The selected rule package keeps the original head, universe and constant policies and replaces its authored roots by the selected instances. Unlisted inherited and delta roots are excluded from this restriction. The signature's stated correspondence between retained witnesses and raw declared computation gives an actual rule morphism into the original declaration calculus. It preserves full formation-sensitive judgments and induces a forward OSLF conversion route. No reverse coverage, normalization or admission theorem is inferred from this inclusion.

Lean. `IndexedFamily.Checking.receiptAt`, `IndexedFamily.Checking.realizeEvidence`, `IndexedFamily.Checking.inclusion`, `IndexedFamily.Checking.judgment_inclusion`, `IndexedFamily.Checking.conversionRoute`

Typed instantiation is another finite check. At a caller's telescope Δ, a substitution request checks formation of Δ and each dependent argument obligation

$$\Delta \vdash \sigma(v) : \sigma(\Gamma_i(v)).$$

The maintained telescope and structural typing checkers perform these checks. Even a zero-argument request must check Δ: an empty argument list cannot authorize an unformed context. Every admitted formation-sensitive context morphism has such finite evidence; this is certificate existence, not an inference procedure.

Lean. `FormationSensitiveChecking.SubstitutionRequest.sound`, `FormationSensitiveChecking.SubstitutionRequest.complete`

With a permitted row, accepted arguments and the exact requested endpoints and displayed type, replay substitutes the original typed receipt. It retains both endpoint typings and the original computation witness at $\sigma(A_i)$. The same request exports a finite raw root code. The reverse implication fails: raw computation does not establish argument typing. Disabling a computation permission also does not construct a propositional identity proof in the smaller calculus.

Lean. `IndexedFamily.Checking.InstanceRequest.replay`, `IndexedFamily.Checking.InstanceRequest.formed`, `IndexedFamily.Checking.InstanceRequest.rootCode_accepted`

The existing natural/vector instance. The cumulative declaration signature has four independently defined iota cases: natural zero and successor, and vector nil and cons. Its actual four canonical schemas now give an exact decoder for that original root relation. The declaration decoder then combines inherited computation, actual delta lookup and these four cases. This supplies the missing executable parameter of common conversion and formation-sensitive judgment checking for the unchanged signature.

Lean. `NativeNaturalVectorFamilies.Checking.iotaDecoder`, `NativeNaturalVectorFamilies.Checking.decoder`, `NativeNaturalVectorFamilies.Checking.conversion_iff_checked`, `NativeNaturalVectorFamilies.Checking.coverage`

The controls check all four raw iota cases, an actual typed natural-eliminator instance in its dependent telescope, and its weakening to a larger context. They reject exchanged dependent arguments,

a changed displayed type, incorrect arities, captured endpoints and disabled rows. An instance with unknown arguments has raw computation evidence but fails the typed instance check. The retained natural iota receipt is reconstructed by the checker, rather than supplied as part of its untrusted request.

Lean. `IndexedFamilyCheckingControls.all_verdicts`,
`IndexedFamilyCheckingControls.checkedZeroReceipt`

These results do not qualify uniform preservation of the raw signature, general positivity checking, or native execution of vector programs. The earlier root-free set comparison does not automatically interpret these added declarations. Natural infinity and no-confusion require their own constructor and induction laws; finite datatype declarations have different consequences. Translation of those laws to HOL Light, Mathdata or configured MM0, and a native realization of this checker, remain separate edges. Partial checking and operational contracts can therefore enter the graph without pretending that every node already discharges every contract.

13.7 Generated rules, bidirectional evidence and dependent append

The simple-inductive generator already produces its recursor and one iota schema per source constructor occurrence. Its computation relation is stated independently of schema replay. The generated ordered table now presents exactly that relation, and the common finite decoder checks the resulting conversion recipes. A receipt retains its constructor ordinal and complete assignment; it translates directly to a single checked root, including below binders. This connection does not identify raw conversion evidence with the generator's annotated typed receipts or establish declaration admission.

Lean. `IndexedPolynomialGenerated.Checking.presents`,
`IndexedPolynomialGenerated.Checking.table_get`,
`IndexedPolynomialGenerated.Checking.receiptCode_accepted`,
`IndexedPolynomialGenerated.Checking.conversion_iff_checked`

The bidirectional producer works against a selected existing rule package. It synthesizes types for variables, declared constants and applications, and checks lambdas and pairs against dependent products and sums. Its inputs include proposed head types, universe joins, normalizations and conversion recipes. These proposals carry no logical authority: every successful submission passes the maintained context and formation-sensitive judgment checkers. Soundness therefore holds even for incorrect producer choices. Fuel bounds the search; refusal is not a proof of nontypability.

Lean. `FormationSensitiveElaboration.replay_sound`,
`FormationSensitiveElaboration.elaborate_sound`

Restricted implicit inference reuses the existing scoped schema matcher. Argument-type patterns share one assignment, so repeated metavariables must agree. Every omitted prefix argument must be determined; no arbitrary value fills an unconstrained slot. The resulting application, with implicit arguments followed by the original explicit arguments, undergoes the same independent checking. For vector cons, the explicit head and tail determine both element type and length. Controls reject an undetermined length, inconsistent repeated element types and missing or extra arguments. This is not a complete higher-order unifier or an instance-resolution algorithm.

Lean. `FormationSensitiveElaboration.Implicit.complete_sound`,
`FormationSensitiveElaboration.Implicit.elaborate_sound`,
`InductiveElaborationControls.inferredSingleton_typed`

Conversion search traverses the maintained syntax and proposes explicit beta, projection, congruence and declaration-root recipes. Each proposed normalization is checked against its original endpoints. Comparing two such results constructs a recipe through their common shape and checks the composite again. The soundness theorems establish conversion, not normal-form existence, completeness of search or forward evaluation of every returned conversion recipe.

Lean. `StructuralConversionSearch.normalize_sound`, `StructuralConversionSearch.compare_sound`

A checked dependent function. The independently authored cumulative Nat/Vec declaration has an element universe parameter u and a motive parameter v. The existing level-instantiation morphism specializes the latter to $\max(u, v)$, retaining both parameters and transporting declaration formation. This licenses the vector-valued motive needed by the following explicit core program:

$$\mathsf{append} : \Pi(A : U_u).\Pi(n, m : \mathsf{Nat}).\mathsf{Vec}\ A\ n \to \mathsf{Vec}\ A\ m \to \mathsf{Vec}\ A\ (\mathsf{add}\ n\ m).$$

Addition uses the declared natural eliminator, and append uses the declared vector eliminator. Neither introduces new recursion or conversion rules. The proof checks the entire closed function at this dependent type in the selected instance, with its schematic element universe and both lengths.

Lean. `NativeNaturalVectorFamilies.Programs.levelInstance`,
`NativeNaturalVectorFamilies.Programs.signature_formed`,
`NativeNaturalVectorFamilies.Programs.judgment_map`,
`NativeNaturalVectorFamilies.Programs.append_typed`

Evidence production runs outside the proof kernel. A shared constructor-data emitter records its finite submission; kernel reduction then checks only that data against the independently written program, type and decoder. The qualified append submission uses 962 shared definitions in 387836 bytes of Lean source. The checker rejects the same submission with its result index changed, its program replaced by the second input vector, or its root typing rule forged. These refusals concern the submitted evidence, rather than general unprovability of other claims.

Lean. `NativeNaturalVectorFamilies.Programs.appendChecked_true`,
`NativeNaturalVectorFamilies.Programs.changedType_refused`,
`NativeNaturalVectorFamilies.Programs.changedTerm_refused`,
`NativeNaturalVectorFamilies.Programs.forgedRoot_refused`

The nil and cons equations of this program hold under arbitrary scoped substitutions. In the cons equation the target contains append applied to the recursive tail; retaining the unprocessed tail instead is refused by the comparison check. These results establish structural conversion, rather than a forward execution strategy or a termination theorem.

Lean. `NativeNaturalVectorFamilies.Programs.Computation.append_substitute`,
`NativeNaturalVectorFamilies.Programs.Computation.append_nil`,
`NativeNaturalVectorFamilies.Programs.Computation.append_cons`

Constructor clauses and checking permissions. A scoped clause compiler consumes the admitted constructor inventory. Its current fragment has one linear, top-level clause per constructor, in source order. The compiler checks names and field counts, replaces written calls on the declared recursive children by induction hypotheses, and constructs methods for the existing eliminator. Capture-safe inverse substitution certifies that replacing those hypotheses by their original calls reconstructs the submitted clause. Whole-term elaboration still uses the independent formation-sensitive checker.

Lean. `ScopedSubstitutionInversion.invert_sound`,
`IndexedInductive.CaseElaboration.compileBranch_reconstructs`,
`IndexedInductive.CaseElaboration.compileMethods_coverage`,
`IndexedInductive.CaseElaboration.elaborate_sound`

For the admitted vector inventory, the written append clauses compile to exactly the independently authored core function above, uniformly in the payload type. Controls refuse missing, extra or reordered clauses, incorrect field counts, calls on another child or index, and a remaining recursive function constant. The concrete source interface includes a noncomputable interpretation of natural indices: this is a kernel-evaluated comparison, not an executable textual frontend. Nested pattern matrices and their compilation remain outside this clause fragment.

Lean. `IndexedInductive.AppendElaboration.methods_match_reference`,
`IndexedInductive.AppendElaboration.compiled_matches_reference`,
`IndexedInductive.AppendElaboration.compiled_typed`,
`IndexedInductive.AppendElaboration.all_verdicts`

The same finite append typing submission also checks in an actual restricted calculus containing just the two natural iota rows, with unchanged heads, declarations and universe instance. Removing either row rejects that submission. The vector rows remain separate permissions for computing vector eliminator applications. Monotone permission inclusions preserve full judgments and their original contexts. Thus a smaller checking basis can be derived from a richer declared judgment, without claiming a globally smallest kernel or unprovability from the rejected submissions. These local results do not establish uniform computational admission of the package.

Lean. `NativeNaturalVectorFamilies.Programs.ComputationProfiles.permissions_mono`,
`NativeNaturalVectorFamilies.Programs.ComputationProfiles.judgment_inclusion`,
`NativeNaturalVectorFamilies.Programs.ComputationProfiles.append_typed_naturalOnly`,
`NativeNaturalVectorFamilies.Programs.ComputationProfiles.all_verdicts`

General source compilation and complete generated-package admission remain separate from that checked target. Acceptance of the corresponding natural laws by native HOL Light, Mathdata and configured MM0, representation retention in CeTTa, source-bound behavioural observations and joined costs remain separate qualification links. No general conservativity, set-model interpretation of the added declarations, or native checker refinement follows from this finite typing result.

One textual program through that checker. The common module reader now consumes one source file containing the Nat and Vec declarations and addition/append constructor clauses. A finite operation adapter resolves names through the maintained nullary-constructor resolver. The separate literal-variable policy retains its own meaning; the adapter does not change general MeTTaIL rewriting. It preserves the complete selected declaration/operation blocks, refusing ignored imports, extra blocks and a different entry.

The compiler's finite plans contain constructor names, field counts and scoped recursive-child calls. Plans derived from the selected textual vector declarations agree with the independently admitted semantic inventory, uniformly in payload type. The source declarations themselves also lower to the original admitted Nat/Vec example. The same clause compiler consumes these plans without evaluating the source's noncomputable natural carrier.

Lean. `IndexedInductive.OperationSource.selectModule_exact`,
`IndexedInductive.CaseElaboration.compilePlan_reconstructs`,
`IndexedInductive.AppendSource.text_admitted`,
`IndexedInductive.AppendSource.source_vector_plans_match`

Named expression expansion preserves the original lexical argument order and commutes with capture-safe substitution. Addition is compiled from its own preceding clauses before append uses it. For this actual source file, the proposed addition and complete append function equal the independent core programs above. The resulting append therefore has the full dependent type, including under the natural-only checking policy. The expected type is the explicit checking query; general dependent function-signature syntax is not supplied by this connection.

Lean. `IndexedInductive.OperationSource.Symbol.instantiate_substitute`,
`IndexedInductive.OperationSource.lower_substitute`,
`IndexedInductive.AppendSource.addition_matches_reference`,
`IndexedInductive.AppendSource.append_matches_reference`,
`IndexedInductive.AppendSource.append_typed_naturalOnly`

The producer remains untrusted. Independent replay checks its actual proposed term rather than accepting a source label or a successful parse. Controls refuse missing/reordered clauses, a call on the wrong recursive child, unbound results and omission of the recursive computation in the submitted dependent evidence. This establishes the selected input's comparison and a general formed-target

soundness boundary, not correctness of this producer for every module. The grammar-driven CeTTa reader and common decoder feed the same Lean checking path; their fixture agreement is distinct from packed parser completeness and from Prime-mode execution with retained representations.

Lean. `IndexedInductive.AppendSource.checkProposal_sound`, `IndexedInductive.AppendSource.all_verdicts`

Joining the informative source package. The generated vector candidate now supplies its actual declaration inventory and two typed schemas to the append checking policy. Joining the independently checked natural prefix recovers the four-row reference table exactly. The candidate's complete declaration entries agree with the independent library; its root judgment and full formed typing judgments include into that library. This route composes with the existing maximum-level instantiation, transporting contexts, terms and result types together. The actual textual append is typed by the resulting generated policy, whose finite executable specialization is proved equal to it.

Lean. `IndexedInductive.AppendChecking.generated_table_reference`, `IndexedInductive.AppendChecking.candidate_extension`, `IndexedInductive.AppendChecking.candidate_judgment_to_program`, `IndexedInductive.AppendChecking.signature_comparison`, `IndexedInductive.AppendChecking.textual_append_typed`

The declaration-source comparison retains the complete generated library, not just the two vector equations. Live grammar-reader proposals now use the same generated checking policy, with only the two natural computation rows enabled. Independent replay checks the actual proposed term; the source fixture's term is not substituted for it. The selected positive reader gate requires exact raw and name-resolved module-data agreement and records the enabled original row ordinals. This is a selected-source admission/checking join, not general parser completeness or a completed native execution chain.

Lean. `IndexedInductive.AppendChecking.textual_candidate_library`, `IndexedInductive.AppendChecking.replayProposal_sound`, `IndexedInductive.AppendChecking.replaySource_sound`

Certificate relocation uses an injective embedding of ordered schema tables, retaining the entire schema and argument list. It checks the original source bound before changing an address, preserves acceptance and refusal under the target permissions, and composes along links. The negative example matters: an invalid local ordinal can name a valid row in the joined table, so merely copying or offsetting an unchecked integer can change its owner. Separately, the common typed-instance checker now has exact finite-certificate coverage for a fixed row and substitution, including target-context formation and independent argument typing. These results do not give uniform computation authority or identify the raw judgment with an annotated typed-equality calculus.

Lean. `SchemaTableConversionCode.Linking.relocate_comp`, `SchemaTableConversionCode.Linking.decode_relocate`, `Declaration.IndexedFamily.Checking.InstanceRequest.checked_arguments_iff`

Annotated instances and the remaining preservation boundary. The same four natural/vector rows now have independently checked annotated endpoint derivations and formed canonical telescopes. Eight declaration types, eight endpoints and four contexts are emitted as ordinary Lean proof source; the receiver checks every proof, so the producer grants no authority. The schema elaborator's inferred variable types and result types agree with these canonical contexts and types. Thus each actual right side satisfies the existing `TemplateTyped` interface.

Lean. `Programs.Annotated.endpoints_typed`, `Programs.Annotated.context_formed`, `Programs.Annotated.template_typed`

If an original row is permitted and its annotated assignment is a typed simultaneous substitution, the maintained premise interface admits the step. Substituting the independent endpoint proofs gives

a typed equality at the instantiated result type. Erasure preserves that equality in the selected raw typed-equality calculus. This does not identify that judgment with the separate formation-sensitive structural-conversion judgment used for append. Each row has a canonical equality witness; altered result-type metadata is refused.

Lean. `Programs.Annotated.instantiated_equal`, `Programs.Annotated.erased_instantiated_equal`, `Programs.Annotated.canonical_equal`, `Programs.Annotated.all_metadataVerdicts`

For this actual package, injectivity and discrimination of annotated type formers suffice for uniform annotated root preservation and admission. Those former facts remain an explicit, undischarged package obligation; the result does not assert unconditional preservation. The natural-only policy also satisfies the existing first-order, left-linear lifting theorem at every annotated input. Vector patterns repeat the element parameter and length index. Their typed instances are covered above, while unrestricted lifting requires a different annotation-coherence argument. Failure of this one syntactic condition is neither a failure of vector checking nor a normalization counterexample.

Lean. `Programs.Annotated.root_preserving_of_formerFacts`, `Programs.Annotated.root_admitted_of_formerFacts`, `Programs.Annotated.lift_natural_root`, `Programs.Annotated.vector_cons_repeats_index`

Actual function annotations and typed natural computation. The natural addition used inside append's result index now has a complete Church-annotated function. Its motive carries the actual natural domain; the recursive method retains the dependent domain obtained by applying that motive. Typed beta equalities supply the conversions in both the method and recursor application. The existing declaration telescope types all arguments independently. Erasure yields the original M1 addition, and the maintained bidirectional annotator reconstructs this independently typed term exactly.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Functions.addFunction_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.producedAddFunction_exact`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_erases`

Its typed zero and successor equations cite the original permitted rows and then compose the actual method's beta equalities. The closed one-plus-zero case is a nonempty witness. With every iota root disabled the function is still typed, while the annotated root relation is empty: typing grants no new computation permission. Two genuinely typed motives with different domains share one erasure. These facts neither infer annotated typing from raw erasure nor discharge general former injectivity.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_zero`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_successor`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.typed_without_iota`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.domain_erasure_distinction`

The complete dependent append consumes this annotated addition. Its original vector constructor and eliminator declarations supply the six dependent arguments; the eliminator is not re-derived. The constructor method retains four explicit argument domains and uses the permitted natural successor equation to convert its constructed vector to the required length. The nil branch uses the permitted zero equation. Abstraction over the independently formed original context gives a closed function at its actual dependent function type, with no package-wide former-facts assumption.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Append.method_body_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.method_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunction_typed`

Erasure identifies both that function and its type with the original raw M1 program. The maintained bidirectional annotator reconstructs the independently typed function exactly when supplied its expected type. Without that guidance, its output still erases to the source but has different domain annotations. The natural-only policy suffices for these finite function derivations; uniform richer-package admission, former facts and unrestricted annotation coherence for the repeated-parameter

vector roots remain distinct obligations.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunction_erases`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunctionType_erases`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.producedAppendFunction_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.unconstrained_erasure_distinction`

The actual annotated append now also has typed nil and cons computation equations. Each cites its original permitted vector row and supplies an independently typed simultaneous assignment. The cons method computes by a general beta law for relative annotated telescopes: its ambient assignment is restricted from the complete typed assignment, while the four method arguments are applied in declaration order. The result retains the substituted dependent type. Closed nil and nonempty-cons examples witness the equations; erasure retains the original raw typed-equality judgment and recursive cons target, without identifying it with formation-sensitive structural conversion.

Lean. `TypedEquality.Annotated.CTele.lams_apply`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.append_nil`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.append_cons`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.closed_cons_erased`

The actual function remains typed under the natural-only policy, which disables both vector rows. Its concrete vector computation certificates are accepted by the full policy and refused by that smaller policy. Function typing, licensed computation and uniform package admission therefore remain separate graph contracts even for the same term and declarations.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Append.typed_with_vector_rows_disabled`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.all_computationVerdicts`

These actual checking policies are connected by the maintained annotated package-morphism interface. A schema-family inclusion with the same declaration table preserves the independently supplied universe-policy map, root instances and their exact premise lists. Morphism composition follows the ancestry of every required premise through both edges. It therefore transports complete typing, equality and subtyping statements, formed contexts and typed substitutions; no proof-size or native-cost conclusion is built into that law.

Lean. `TypedEquality.Annotated.ChurchRules.ofSchemas_morphism`, `TypedEquality.Annotated.ChurchRules.Morphism.comp`, `NativeNaturalVectorFamilies.Programs.Annotated.judgment_mono`, `NativeNaturalVectorFamilies.Programs.Annotated.context_mono`, `NativeNaturalVectorFamilies.Programs.Annotated.substitution_mono`

For the actual append, the natural-only policy includes into an intermediate policy that additionally licenses vector nil, and thence into all four original rows. The independently typed function crosses this composed route. Its closed nil equation already holds at the intermediate node; its actual cons certificate refuses there and accepts at the full node. These are computation policies over the same declared calculus, not complete Lean or extensional profiles and not a reverse theorem-transport result.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Append.function_after_two_extensions`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_before_cons`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.cons_certificate_strict`

The authored natural carrier and HOL formulas. The sequentially admitted natural declaration has an independently established representation by ordinary naturals. A new consumer of its original polynomial eliminator gives the function satisfying

$$f(\mathsf{Zero}) = z, \qquad f(\mathsf{Succ}\ n) = s\ n\ (f\ n),$$

and proves uniqueness and full predicate induction. Its actual zero and successor therefore supply the existing Peano structure, including injectivity, disjointness and an infinite carrier. Lifting that

carrier to the universe of the maintained HOL model constructs a full-domain interpretation of the existing Peano formulas. For every target simple type, an explicit HOL formula for the existence and uniqueness of the recursive function is true in this model. The proof uses the source eliminator rather than postulating a recursion axiom.

Lean. `IndexedInductiveNaturalHOL.recursion_exists_unique`, `IndexedInductiveNaturalHOL.models_axioms`, `IndexedInductiveNaturalHOL.models_recursion`, `IndexedInductiveNaturalHOL.infinite_carrier`

This connects the declared source carrier to actual HOL formulas, not merely to a list of names for natural-number facts. It is a particular semantic interpretation: it neither extracts proofs from truth in one model nor identifies the model's constants with a serialized HOL Light closure. The selected native HOL Light law interface below now supplies exact statement comparison; Mathdata/MM0 transport and full source definition-history refinement retain their own obligations. The source's noncomputable semantic recursor supplies no executable numeric representation or performance result.

The selected native HOL Light natural-law interface. Five independently typed intrinsic formulas now retain the actual source presentation: numeral wrapping, the result-before-predecessor recursion method, Boolean equality in successor injectivity, and an explicit injective endomorphism with an omitted value for carrier infinity. The recursion root is existential; uniqueness belongs to the stronger source theorem above. The actual authored carrier interprets all five formulas at arbitrary result carriers using its original eliminator. A Boolean-valued instance computes distinct results at zero and successor. A finite collapsing model refutes no-confusion, and removing the source's numeral wrapper changes the native syntax even though its constructed interpretation reads that wrapper as identity.

Lean. `HOL.NaturalFacts.recursion_meaning`, `HOL.NaturalFacts.infinity_meaning`, `HOL.NaturalFacts.collapsing_noConfusion_refused`, `IndexedInductiveNaturalHOL.NativeFacts.all_models`, `IndexedInductiveNaturalHOL.NativeFacts.boolean_recursion_witness`, `HOLLight.NaturalFacts.bare_zero_statement_distinct`

The existing source exporter produces the four retained natural roots and a new carrier-infinity theorem derived from successor injectivity and no-confusion. Native CeTTa replay checks the 23225-node closure as 67384 records and returns all five statements exactly as printed from the independent typed terms. The declared standard HOL policy retains its three axioms; no source axiom is added. Altered claims, an additional axiom and truncation refuse natively, while missing, changed and repeated statement packets refuse at the intake gate. Production, adapter and replay costs remain distinct. The source producers are untrusted. This qualifies the selected law-signature interface; full polymorphic source definition-history refinement and linked-C correctness are not inferred.

Lean. `HOLLight.HOLSyntaxWire.encodeTerm_rename`, `HOL.Names.lift_comp`

An axiom-free native closure component. The Megalodon definition

$$N_{z,s}(n) \;:=\; \forall P.\; P(z) \to (\forall k.\; P(k) \to P(s(k))) \to P(n)$$

uses only simple types, implication and quantification. Its native Mathdata body and the zero, successor and induction statements erase exactly from independently typed HOL terms, after the declared definition is unfolded. The finite zero and successor proof objects also pass the existing executable Mathdata checker in Lean. A separate Megalodon source file proves these laws, including induction with a closure hypothesis in the step. Native CeTTa replay accepts all six source proofs with an empty imported-axiom allowlist and compares the exact definition and three principal statements. Both CeTTa dialects agree on these observations. Altered conclusions and unbound hypotheses refuse; failed claims are not published.

Lean. `Megalodon.NaturalClosure.body_erases`,
`Megalodon.NaturalClosure.induction_statement_erases`,
`Megalodon.NaturalClosure.zero_proof_checked`,
`Megalodon.NaturalClosure.successor_proof_checked`

The selected authored natural carrier satisfies this very predicate at every value. Its existing induction removes the extra closure hypothesis without excluding any source values. Interpreting successor as constant zero instead excludes a genuine source successor, while still admitting zero; this is a nonempty negative comparison. Closure and induction alone do not imply successor injectivity, zero separation, unique recursion or infinity. Those native-route obligations remain explicit, and no proof is extracted from truth in the particular source model.

Lean. `IndexedInductiveNaturalMegalodon.source_member`,
`IndexedInductiveNaturalMegalodon.source_native_formulas`,
`IndexedInductiveNaturalMegalodon.collapsing_successor_refused`

The same closure through MM0's HOL presentation. The typed closure syntax and its Henkin meaning now have a common HOL owner. An MM0 syntax producer retains types and explicit binder names; it commutes with intrinsic variable renaming. The natural-closure definition body printed from that typed term supplies a second specification for the actual MM1 proofs. Native MM0 accepts zero closure, successor closure and ordinary closure elimination against both specifications, with their original object-typing hypotheses. The latter rule uses an unrestricted predicate step; it is distinct from the strengthened induction statement above.

Lean. `MM0.HOLSyntaxWire.encodeTerm_rename`, `MM0.NaturalClosure.definition_meaning`

CeTTa's HE, PeTTa and Prime modes each pass fourteen exact-statement, retained-proof and refusal controls on the same artifact. A separately built upstream `mm0-c` also accepts it and refuses changed definitions and conclusions. The source extension adds no axioms. The producer's dependency audit reports only logical, typing and equality rules, without choice, eta, type-definition or infinity axioms. Those axioms remain available in the loaded full HOL presentation; extracting a smaller base is separate work. A finite collapsing interpretation admits zero but excludes another inhabitant, showing why closure alone does not supply infinity. General HOL-presentation adequacy and full native natural-number fact transport remain open.

Lean. `MM0.NaturalClosure.finite_zero`, `MM0.NaturalClosure.finite_other_refused`

Closure-premised induction in the same MM0 route. The next source extension proves induction with the generated-number guard retained in its step premise. It strengthens the property to the conjunction of closure membership and the property, applies the existing ordinary elimination rule, then extracts the property. This is a derived proof using the existing logical core, with one auxiliary definition and no new axioms. An independent public statement and definition specification are emitted through the maintained MM0 wire machinery. The compiler and format producer remain outside the theorem authority.

The corresponding intrinsic formula holds in every chosen Henkin model. For admitted zero, successor and property, the strengthened predicate is the denotation of an actual lambda term. Henkin term closure therefore admits this particular predicate without a full-domain assumption. In a three-point interpretation, zero maps to itself while an unreachable point satisfying the property maps to a point that fails it. Its guarded step and induction hold, while the unguarded step fails. This separates the rule premises, not the strength of any dependent candidate.

Lean. `HOL.NaturalClosure.parameterValuation_admissible`,
`HOL.NaturalClosure.denote_strengthening`, `HOL.NaturalClosure.strengthening_admitted`,
`HOL.NaturalClosure.closedUnder_induction_admitted`,
`HOL.NaturalClosure.models_induction_henkin`,
`HOL.NaturalClosure.InductionControls.closed_iff_zero`,
`HOL.NaturalClosure.InductionControls.guarded_step`,

`HOL.NaturalClosure.InductionControls.unguarded_step_refused`,
`HOL.NaturalClosure.InductionControls.guarded_induction`

Upstream verification accepts the compiled artifact against both source and independent specifications. Native HE, PeTTa and Prime modes each pass twenty controls in both the selected optimized and instrumented configurations, using the same 65,328-byte artifact. Replayed induction retains all seven premises; changed guards, dropped membership and exchanged hypotheses are refused. Changed auxiliary definitions, changed conclusions and truncation are refused as well. The dependency inventory uses logical, typing and equality rules without choice, eta or infinity. The original closure profile still passes fourteen controls per mode through the same gate. These are qualified proof/statement routes; full natural recursion, no-confusion, infinity and source-presentation adequacy remain separate obligations.

Conditional native unique recursion. The extension defines the least graph containing (z, a) and closed under $(n, v) \mapsto (s(n), f(n, v))$. Its independent inductive reading is functional when s is injective and never returns z. Every generated number has a graph value. A selector satisfying its explicit selection law therefore defines a recursive function, unique on the natural closure. The native statement retains all three interfaces; none is an imported natural-number axiom.

Lean. `Megalodon.NaturalRecursion.Graph.functional`,
`Megalodon.NaturalRecursion.Graph.exists_of_generated`,
`Megalodon.NaturalRecursion.selected_unique`

In full-domain interpretations, the intrinsic HOL graph denotes exactly this inductive graph. The independent typed recursion statement has the stated zero/successor equations and pointwise uniqueness. Its native erasure is well formed, and the retained native proof is checked against that erasure. The combined source supplies 24 proofs and seven definitions, with no imported axioms; replay, portable evidence and altered-claim controls pass in both CeTTa dialects. Compiled Lean checking also accepts the complete ordered source batch. This last execution result is not a kernel-reduced proof of the finite batch-acceptance proposition.

Lean. `Megalodon.NaturalRecursion.denote_graph_iff`,
`Megalodon.NaturalRecursion.models_recursion`,
`Megalodon.NaturalRecursion.native_statement_erases`,
`Megalodon.NaturalRecursion.native_statement_formed`

On the authored carrier, every graph value equals the original eliminator's value. All valid selectors agree there, and a classical interpretation supplies a selector at every inhabited target. This is a semantic choice witness, not a runtime provider or a global dependent-type-theory option. A constant-zero selector fails a genuine successor computation. The full native no-confusion/infinity interface and MM0 transport remain open.

Lean. `IndexedInductiveNaturalMegalodon.source_graph_iff`,
`IndexedInductiveNaturalMegalodon.source_selectors_agree`,
`IndexedInductiveNaturalMegalodon.source_recursion_with_choice`,
`IndexedInductiveNaturalMegalodon.constant_selector_wrong_result`

The selected structural-conversion node. The actual natural-only checking policy retains original schema occurrences 0 and 1. Its two independent constructor schemas agree exactly with that selection. Their left sides are linear constructor patterns, their right-side variables are covered, and two matching left sides determine the same rule. The maintained constructor-system theorem therefore proves confluence of the policy's raw beta/iota conversion, including pair projections and its declared head equality. It also supplies the Pi/Sigma conversion boundaries used by formation-sensitive preservation. Addition at zero witnesses actual computation. Conversely, no conversion identifies the declared natural type with a Pi, including paths outside its simple image. These results do not supply termination, annotated typed-eta former facts, or a constructor-system presentation for the repeated-parameter vector schemas.

Lean. `ComputationProfiles.NaturalConversion.schema_iff`,
`ComputationProfiles.NaturalConversion.roots_exact`,
`ComputationProfiles.NaturalConversion.churchRosser`,
`ComputationProfiles.NaturalConversion.pi_boundary`,
`ComputationProfiles.NaturalConversion.sigma_boundary`,
`ComputationProfiles.NaturalConversion.addition_zero`,
`ComputationProfiles.NaturalConversion.natural_not_pi`

Uniform preservation in the checking policy. The declaration-telescope interface recovers the motive, zero method, successor method and input from any typing of the actual recursor call. Recovery retains the original formed declarations and replays its complete conversion/cumulativity tail. At zero, the recovered zero method already has the principal result type. At successor, constructor-argument recovery types the predecessor; renewing the input component of the typed substitution constructs the recursive call. The same successor method then supplies the contractum. The shared telescope interface now proves construction as well as recovery, uniformly in its finite dependent arity.

Lean. `FormationSensitive.DeclarationSpine.applyTelescope`,
`ComputationProfiles.NaturalConversion.Preservation.recover`,
`ComputationProfiles.NaturalConversion.Preservation.zero_preserves`,
`ComputationProfiles.NaturalConversion.Preservation.successor_preserves`

These root proofs and the earned conversion boundaries discharge contextual preservation for this checking policy. The universe and head laws are inherited from the tower. Beta reduction, declared natural computation and projections preserve the formed context and exact displayed type under every context. Concrete checked zero and successor redexes witness the premises; four actual steps compute the latter to the successor of zero. Reusing its submitted evidence with a Pi result is refused. The policy still disables vector roots. Richer annotated typed-eta former facts remain distinct from the actual whole-function annotation already checked above.

Lean. `ComputationProfiles.NaturalConversion.Preservation.roots`,
`ComputationProfiles.NaturalConversion.Preservation.steps_preserve`,
`ComputationProfiles.NaturalConversion.Controls.zero_typed`,
`ComputationProfiles.NaturalConversion.Controls.successor_path`,
`ComputationProfiles.NaturalConversion.Controls.successor_preserved`,
`ComputationProfiles.NaturalConversion.Controls.altered_type_refused`

Operational roots with a smaller checking basis. All four original operational iota roots preserve this same smaller checking judgment. Recovering the vector eliminator's six dependent arguments retains its original declaration and displayed-type adjustment. The nil target is its recovered branch. In the cons case, a second telescope inversion recovers the constructor's length, head and tail. They extend the recovered parameter substitution into the original canonical context. The canonical contractum uses the existing eliminator and retains its recursive call; substitution and replay then give the exact original displayed type. Closed formation of that eliminator is recovered from the actual source derivation, without assuming preservation for the larger operational relation.

Lean. `FormationSensitive.Typing.applyClosed_head`,
`FormationSensitive.DeclarationSpine.ofObservedTelescope`,
`FormationSensitive.DeclarationSpine.constant_formed`,
`ComputationProfiles.NaturalConversion.Preservation.Vector.recover`,
`ComputationProfiles.NaturalConversion.Preservation.Vector.constructor_arguments`,
`ComputationProfiles.NaturalConversion.Preservation.Vector.canonical_cons_target`,
`ComputationProfiles.NaturalConversion.Preservation.Vector.nil_preserves`,
`ComputationProfiles.NaturalConversion.Preservation.Vector.cons_preserves`,
`ComputationProfiles.NaturalConversion.Preservation.operational_roots`,
`ComputationProfiles.NaturalConversion.Preservation.operational_rule_roots`

The canonical nil and cons sources are independently checked under the smaller policy, and the

new theorem types their operational targets. Nevertheless the cons rule is not a root conversion of that checker: its concrete certificate is refused there and accepted with the original full permissions. This distinguishes execution from checking authority. The theorem covers roots; lifting a larger operational relation through arbitrary dependent contexts requires an additional transport law, since its steps are not thereby available as checking conversions.

Lean. `ComputationProfiles.NaturalConversion.Controls.vector_nil_typed`, `ComputationProfiles.NaturalConversion.Controls.vector_cons_typed`, `ComputationProfiles.NaturalConversion.Controls.vector_nil_preserved`, `ComputationProfiles.NaturalConversion.Controls.vector_cons_preserved`, `ComputationProfiles.NaturalConversion.Controls.vector_cons_root_disabled`, `ComputationProfiles.NaturalConversion.Controls.vector_cons_citation_refused`, `ComputationProfiles.NaturalConversion.Controls.vector_cons_citation_enabled`

The opaque annotated baseline. Disabling every declared iota permission leaves all eight original Nat/Vec declarations and their level/head policy intact. The maintained theorem for packages without root computation gives the richer annotated former injectivity and no-confusion laws. In particular, the actual addition function's product type cannot equal a universe. The same function is typed with those permissions disabled; its actual first beta application is an equality at its displayed function type.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.inactive_raw_no_root`, `NativeNaturalVectorFamilies.Programs.Annotated.inactive_formerFacts`, `NativeNaturalVectorFamilies.Programs.Annotated.inactive_pi_not_head`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_function_type_not_universe`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_function_beta`

The cumulative head-preservation argument has a common owner and serves both this declaration policy and the existing set-profile client. Along with the earned opaque former facts, it discharges the maintained contextual preservation theorem: every finite beta/projection/head reduction of a term typed in a formed context gives a typed equality, and a term at the exact original dependent type. No termination is assumed or concluded. The independent annotator still reconstructs the real function. Its natural-zero root citation is refused here and accepted when the original row is enabled. The absence of declared roots is explicit; this baseline does not admit an enabled Nat/Vec computation or establish annotation coherence.

Lean. `Annotated.CHeadPreserving.ofLevelTower`, `NativeNaturalVectorFamilies.Programs.Annotated.head_preserving`, `NativeNaturalVectorFamilies.Programs.Annotated.inactive_reduction_equal`, `NativeNaturalVectorFamilies.Programs.Annotated.inactive_reduction_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_zero_citation_refused`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.full_zero_citation_accepted`

The natural annotation has a nontrivial reading. The actual natural type, zero, successor and dependent recursor now use the existing domain numeral ideals and type-projected constants. Staging constant lookup avoids a recursive definition of the interpretation. The four original declared types agree with the maintained numeral interface. All eight declarations remain present; the other constants are opaque here and their computation is disabled.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.nat_declared`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.zero_declared`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.successor_declared`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.recursor_declared`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.constants_valid`

Every enabled root is an instance of one of the original two natural rows. Its actual annotated argument spine supplies the dependent semantic telescope premises; the interpretation then uses the existing projected zero and successor computation theorems. Result-type and spine premises are

retained. Together with the original head policy this gives a valid reading of the selected annotated package. The existing closed proof of one plus zero denotes successor of zero. That value is distinct from zero, and soundness refutes a purported annotated equation between them. These are concrete semantic and negative controls for this package, not a new strength separation or a general claim about dependent candidates. Semantic validity does not supply logical-relation adequacy, enabled former injectivity, unrestricted annotation coherence or native refinement.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.recursor_spine`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.zero_valid`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.successor_valid`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.roots_valid`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.valid`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.one_plus_zero_value`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.zero_ne_successor_zero`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.zero_equal_successor_refused`

An opaque decoder extension that retains computation. The maintained package sum adds the existing fresh proposition/decoder declarations to the enabled natural annotation. Spine facts are contravariant in declaration inclusion: a larger signature's facts retain each original declaration's type. Consequently a valid reading of the original package and a valid reading of an opaque second package give a valid reading of their sum. Original constants keep their types, and every root comes from the original package; the second has none. This common semantic law also handles shadowed second declarations.

Lean. `Annotated.SpineFacts.of_decl_inclusion`, `Annotated.ReadingValid.sum_opaque`

The concrete decoder names are fresh, and its two declarations have their stated types. Both original natural roots remain with precisely their original required evidence. An actual zero computation and the typed one-plus-zero proof survive the existing forward judgment map, while the false zero/successor equation is still refuted. The opaque decoder constants have no observations in this reading; the natural constructors and recursor retain their nontrivial interpretation. This is neither the no-root extension used for opaque base packages. The forward semantic law concerns validity; the return comparison below supplies conservativity for its stated fragment. Recursor adequacy and enabled former facts remain further obligations.

Lean. `NaturalDomain.Decoder.fresh_prop`, `NaturalDomain.Decoder.fresh_holds`, `NaturalDomain.Decoder.prop_declared`, `NaturalDomain.Decoder.holds_declared`, `NaturalDomain.Decoder.declarations_valid`, `NaturalDomain.Decoder.valid`, `NaturalDomain.Decoder.roots_iff`, `NaturalDomain.Decoder.premises_iff`, `NaturalDomain.Decoder.zero_root`, `NaturalDomain.Decoder.one_plus_zero`, `NaturalDomain.Decoder.zero_equal_successor_refused`

The return route preserves the required evidence. The existing constant-instance interface maps the fresh proposition to its universe and the decoder to the identity at that universe. Original declared names and all eight closed declared types stay fixed. The templates, inferred variable types and reflexivity equations of all four original rows satisfy the corresponding support conditions. Hence the comparison is uniform over their permission selections; repeated vector parameters do not require an annotation-lifting claim for this operation.

Lean.
`NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.original_names_fixed`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.declarationTypeFixedCheck_true`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.original_types_fixed`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.row_left_fixed`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.row_right_fixed`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.row_variable_type_fixed`, `NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.schema_support`

The common schema law lets arbitrary instance arguments change under the constant interpretation. Both endpoints remain instances of the original schema, and the entire required-premise list becomes precisely the list for the changed assignment. Root stability alone would not establish that law. Original constant formation, these schema laws and the maintained decoder-definition typing proofs construct the full return interpretation.

Lean. `Annotated.patternPremises_instConsts_of_fixed`, `Annotated.SchemaConstantsFixed.root`, `Annotated.SchemaConstantsFixed.requires`, `Annotated.ChurchRules.ofSchemas_constInstance`, `NaturalDomain.Decoder.original_interpretation`, `NaturalDomain.Decoder.decoder_type_expansion`, `NaturalDomain.Decoder.return_interpretation`, `NaturalDomain.Decoder.returned_judgment`

For every statement fixed by this interpretation, derivability in the extended package is equivalent to derivability in the original package. The fixed-statement condition covers context, displayed type and all terms. The actual addition statement reflects at both natural-only and full permissions. Conversely the fresh proposition's typing statement changes to the typing of the universe, so the interpretation is not the identity. The false natural constructor equation also refuses via the return route. These are theorem-level judgment maps. No equality of retained proof data, serialized certificate translator, annotation-coherence result or execution cost bound follows from them.

Lean. `NaturalDomain.Decoder.fixed_statement_iff`, `NaturalDomain.Decoder.proposition_typed`, `NaturalDomain.Decoder.returned_proposition_typed`, `NaturalDomain.Decoder.proposition_statement_changes`, `NaturalDomain.Decoder.original_addition_fixed`, `NaturalDomain.Decoder.original_addition_reflected`, `NaturalDomain.Decoder.full_policy_addition_reflected`, `NaturalDomain.Decoder.false_equation_refused_by_return`

Typing the supplied HOL type-definition equation. For the displayed declarations $\mathsf{abs} : A \to N$, $\mathsf{rep} : N \to A$ and $a : N$, the term $\mathsf{abs}(\mathsf{rep}(a))$ has type N. The maintained OpenTheory source checker accepts its equality with a at N, and rejects an equality indexed by A whenever $A \neq N$. This is proved for arbitrary types, with concrete distinct-type controls.

Lean. `MM0.HOLTypeDefinitionBoundary.roundtrip_typed`, `MM0.HOLTypeDefinitionBoundary.defined_equality_typed`, `MM0.HOLTypeDefinitionBoundary.representation_equality_refused`, `MM0.HOLTypeDefinitionBoundary.defined_equality_checked`, `MM0.HOLTypeDefinitionBoundary.representation_equality_check_refused`, `MM0.HOLTypeDefinitionBoundary.distinct_types`, `MM0.HOLTypeDefinitionBoundary.all_verdicts`

The pinned supplied `hol.mm0`/`hol.mm1` example had the representation-type index in `abs\_rep`; its native declaration confirmed that exact source formula. Both local files now use the defined-type index. Qualification compares the complete native axiom declaration with an independent reference and refuses the reverted specification using both the separate verifier and native service. The argument and hypothesis inventories are retained; no axiom is added. The qualified closure/induction proofs do not depend on this axiom. This repair addresses source typing, without inferring inconsistency of the earlier untyped presentation or full HOL-presentation adequacy.

Authorization retains the selected judgment. The legacy indexed-family authorization interface requires uniform `HasType` preservation. The current formation-sensitive checker uses a judgment containing its formed ambient context and additional formation premises; the annotation has a further typing/equality interface. Forgetting refined derivations gives raw ones. A converse upgrade from arbitrary raw derivations is impossible even with a formed empty context: the raw identity rule permits an undeclared domain, whereas regularity and product inversion of the refined judgment require that domain's declaration. This comparison is proved generically and checked for every original Nat/Vec permission selection. The actual append remains a positive refined derivation with its raw view.

Lean. `FormationSensitive.raw_identity_at_any_domain`, `FormationSensitive.judgment_identity_domain_declared`, `FormationSensitive.raw_identity_missing_domain`, `FormationSensitive.no_uniform_raw_upgrade`

These are deliberately different specified judgments. This does not refute legacy preservation or expose a strength limitation of a dependent candidate. It identifies the remaining interface obligation: the proved formed operational preservation and annotated former-fact conditional cannot alone fill the legacy raw-authorization field. That field needs its own qualified proof or an explicit, judgment-indexed admission route; no weakening or identification of those interfaces is assumed here.

A concrete pointed injection in the native MM0 route. The next proof extension selects an injective, non-surjective endomorphism of the supplied HOL individual type and a point outside its image. Its six public results prove the successor and zero typings, successor injectivity and non-surjectivity, and omitted-zero/no-confusion statements. They use only the existing logical rules, choice, eta and infinity; no natural-number fact is supplied as a new axiom. Intrinsically typed definition bodies and an independent statement emitter bind this extension to the maintained MM0 specification. The omitted-point formula has its stated Henkin meaning without a full-function-domain hypothesis.

Lean. `MM0.NaturalCarrier.noPreimage_meaning`, `MM0.NaturalCarrier.all_verdicts`

The separate verifier and native service accept the same compiled extension against the independent specification. Altering the injection/zero constructions, reversing no-confusion, changing zero's type, or removing the typed argument from proof replay is rejected. Both selected optimized and sanitizer configurations pass 36 controls in each dialect on the same 73,808-byte artifact. The unchanged closure/induction profiles retain their 16/22 controls and original artifact identities. The source's declaration order is part of this checked interface. This establishes a pointed infinite carrier; forming its natural subtype and proving primitive recursion and complete source-to-native natural facts remain required. Neither the MM1 compiler nor the entire supplied HOL presentation is thereby formally refined.

The native natural subtype. The existing nonempty-predicate type-definition principles form the subtype of the selected zero/successor closure. Representation and abstraction transport constructor typing, zero/successor separation, successor injectivity, induction and Dedekind infinity. The induction proof reuses the established closure theorem through an explicit predicate pullback; no new natural-number axiom or authored eliminator is introduced. The abstraction/representation equality is used at the actual defined type. Its exact native proof dependencies include the existing four type-definition principles. The induction proof does not depend on the infinity axiom; constructor separation and infinity retain that dependency. These are proof-local dependency statements, not an automatic weakening of the ambient specification.

Lean. `MM0.NaturalType.all_verdicts`

An independent reference specifies the subtype, constructors, pullback and six public statements. The shared gate rejects altered membership, pullback, equality indices, missing induction premises and switched replay inputs. Both selected native configurations pass 50 controls per dialect on the same 87,400-byte artifact. The native Boolean-equality abbreviation is retained in exact statement comparison. This is a checked natural-subtype construction, not yet primitive recursion, a complete comparison with the authored Nat interpretation, or a refinement of the compiler/native C.

14 Status ledger

Entries marked "proved" are checked without `sorry`, custom axioms or native decision. Entries marked "open" are obligations, not proved claims. Axioms are those reported by `##print axioms`

on the listed declaration: P = `propext`, C = `Classical.choice`, Q = `Quot.sound`; "none" means no axioms.

Part	Claim	Lean declaration	Axioms	Status
A1	Freeness: multifunctors out of open derivations are interpretations of rule instances	`RuleSignature.universal`	Q	proved
A1	A multifunctor is determined by its generators	`RuleSignature.functor_eq_of_gen`	Q	proved
A1	Extension agrees with the interpretation on generators	`RuleSignature.lift_gen`	none	proved
A1	Extension preserves substitution	`RuleSignature.Term.lift_subst`	Q	proved
A1	Right unit law	`RuleSignature.Term.subst_vars`	Q	proved
A1	Associativity of substitution	`RuleSignature.Term.subst_subst`	Q	proved
A1	Closed derivability is the inductive rule closure	`RuleSignature.derivable_iff_derives`	PQ	proved
A1	Closed derivations are planar	`RuleSignature.Term.closed_linear`	P	proved
A1	Generators are planar	`RuleSignature.Term.gen_linear`	PQ	proved
A1	Holes of equal sort stay distinct	`RuleSignature.Term.duplicate_holes_distinct`	P	proved
A1	Stable instance actions induce multifunctors	`RuleSignature.actFunctor`	PQ	proved
A1	Derivability is closed under instance actions	`RuleSignature.Derivable.act`	PQ	proved
A1	Substitution acts on Hilbert multicategories	`Hilbert.substFunctor`	PQ	proved
A1	Hilbert derivability is substitution closed	`Hilbert.source_subst`	PQ	proved
A1	Instance actions compose coherently	`RuleSignature.act_comp_heq`	PQ	proved
A1	Substitution multifunctors compose coherently	`Hilbert.substFunctor_comp_heq`	PCQ	proved
A1	Multifunctors compose associatively	`CartesianMulticategory.Functor.comp_assoc`	Q	proved
A1	Multifunctors preserve closed morphisms	`CartesianMulticategory.Functor.inhabited`	none	proved
A1	Kernel derivability is closed derivability	`RuleWitness.derivable_iff_derives`	PQ	proved
A1	Closed morphisms are accepted certificate trees	`RuleWitness.closedEquiv`	PQ	proved
A1	Erasure to trees is injective	`RuleWitness.toDerivation_injective`	PQ	proved
A1	Unfolding replay agrees with judgment replay	`RuleWitness.unfold_judgments`	PQ	proved
A1	Every accepted graph node unfolds to a closed morphism	`RuleWitness.unfold_root`	PQ	proved
A1	Every accepted graph node yields an accepted tree	`RuleWitness.unfold_tree`	PQ	proved
A1	Every closed morphism is the unfolding of an accepted graph root	`RuleWitness.presented`	PCQ	proved
A1	Presented blocks combine into one graph	`RuleWitness.presented_block`	PCQ	proved
A1	Authored open derivations are free-multicategory terms	`CertificateGSLT.OpenDerivation.termEquiv`	PQ	proved
A1	Plugging is substitution	`CertificateGSLT.OpenDerivation.toTerm_bind`	PQ	proved
A1	Interpretation of open derivations preserves substitution	`CertificateGSLT.interpret_bind`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
A1	Interpretation of open derivations is unique	`CertificateGSLT.interpret_unique`	PQ	proved
A1	Computable decoding of accepted raw open proofs	`CertificateGSLT.decodeOpen`	PCQ	proved
A2	Translations preserve derivability	`RuleSignature.Derivable.map`	Q	proved
A2	Injective renamings induce faithful multifunctors	`RuleSignature.lift_rename_injective`	Q	proved
A2	Intuitionistic into classical: faithful	`Hilbert.inclusion_faithful`	PQ	proved
A2	Intuitionistic into classical: not reflecting	`Hilbert.inclusion_not_reflecting`	PCQ	proved
A2	Classical into extended: faithful	`Extension.inclusion_faithful`	PQ	proved
A2	Classical into extended: conservative	`Extension.inclusion_conservative`	PCQ	proved
A2	Classical into extended: not full	`Extension.inclusion_not_full`	PCQ	proved
A2	HOL Light into its first-order calculus: multifunctor	`HOLLight.FirstOrder.translation`	PCQ	proved
A2	That multifunctor is determined by rule images	`HOLLight.FirstOrder.translation_unique`	PCQ	proved
A2	It preserves derivability	`HOLLight.FirstOrder.translation_derivable`	PCQ	proved
A2	It reflects derivability	`HOLLight.FirstOrder.translation_reflects`	PCQ	proved
A2	Positive control: REFL translated	`HOLLight.FirstOrder.Kernel.reflexivity_translated`	PCQ	proved
A2	Negative control: F not derivable	`HOLLight.FirstOrder.Kernel.falsity_not_translated`	PCQ	proved
A2	Node-by-node translation of one node is the multifunctor image	`RuleWitness.liftStep_liftEntry`	PCQ	proved
A2	Shared translation commutes with unfolding	`RuleWitness.replay_liftStep`	PCQ	proved
A2	Accepted graphs translate with sharing, coherently	`RuleWitness.shared_translation`	PCQ	proved
A2	HOL Light graphs translate with sharing, coherently	`HOLLight.FirstOrder.shared_translation`	PCQ	proved
A3	Logical work is additive under substitution	`RuleSignature.Term.size_subst`	PQ	proved
A3	Affine translations are linear on trees	`RuleSignature.lift_size_le`	PCQ	proved
A3	Contraction: linear source, exponential tree, linear graph	`Contraction.contraction_gap`	PCQ	proved
A3	The contracting image is not affine	`Contraction.image_not_affine`	PQ	proved
A3	Trees do not size-simulate graphs (existing)	`ProofSharingBounds.no_tree_sizeSimulation_dag`	PCQ	proved
A3	Literal expansion of definitions is exponential (existing)	`PropositionalExtension.literal_expansion_gap`	PCQ	proved
A3	HOL graph to MM0 certificate bound, degree 10 (existing)	`HOLLight.FirstOrder.graphTranslate_weight`	PCQ	proved
A3	Declared time is at least the written lemma weight	`HOLLight.FirstOrder.graphTime_ge_weight`	PCQ	proved
A3	MM0 p-simulates HOL Light graphs, declared time (existing)	`HOLLight.FirstOrder.Kernel.mm0Simulation`	PCQ	proved
A3	Operational clocks are positive (existing)	`ProofSystems.OperationalPSimulation.time_positive`	PCQ	proved
A4	Sequential replay is local	`DerivationGraph.replayWith_iff_local`	PCQ	proved
A4	Schedule independence (forward)	`DerivationGraph.replayWith_permute`	PCQ	proved
A4	Schedule independence (reflection)	`DerivationGraph.replayWith_permute_reflect`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
A4	Schedule independence (both directions)	`DerivationGraph.replayWith_permute_iff`	PCQ	proved
A4	Dependency-violating schedules are refused	`DerivationGraph.replayWith_permute_refused`	PCQ	proved
A4	Lookup count is schedule invariant	`DerivationGraph.permute_lookups`	PCQ	proved
A4	Replayed blocks relocate into larger caches	`DerivationGraph.replayWith_relocate`	PCQ	proved
A4	Unfoldings are schedule invariant	`RuleWitness.unfold_permute`	PCQ	proved
A4	Monotone single-product steps are graph replay	`MultiPremiseSteps.run_monotone`	PQ	proved
A4	Monotone traces denote closed morphisms	`MultiPremiseSteps.run_monotone_unfolds`	PQ	proved
A4	Monotone multi-product steps split	`MultiPremiseSteps.run_split`	PQ	proved
A4	Consumption is linear	`MultiPremiseSteps.consumed_not_cited`	PQ	proved
A4	Consumption is not confluent	`MultiPremiseSteps.Controls.consumption_not_confluent`	PQ	proved
A4	Independent consuming steps commute up to product order	`MultiPremiseSteps.independent_commute`	PQ	proved
A4	Control: two independent consuming steps commute	`MultiPremiseSteps.Controls.independent_steps_commute`	PQ	proved
A4	Split/join exceeds tree-shaped single-wire-cut composition	`MultiPremiseSteps.process_not_polycategorical`	PQ	proved
C	HOL Light kernel module loads exactly	`HOLLight.FirstOrder.kernelModule_read`	PCQ	proved
C	Loaded HOL Light module is faithful	`HOLLight.FirstOrder.kernelModule_faithful`	PCQ	proved
C	VibeITP module loads exactly (existing)	`VibeITP.CheckingMachine.kernelModule_read`	PCQ	proved
C	RUP instances are sound	`PropositionalLRATActions.rupCheck_sound`	PQ	proved
C	LRAT action checker is sound	`PropositionalLRATActions.check_sound`	PQ	proved
C	LRAT actions denote closed morphisms	`PropositionalLRATActions.check_unfolds`	PQ	proved
C	RAT additions are not RUP rule instances	`PropositionalLRATActions.rat_not_rule_instance`	PQ	proved
C	Positive control: RUP refutation	`PropositionalLRATActions.Controls.xor_accepted`	PQ	proved
C	Negative control: deleted premise	`PropositionalLRATActions.Controls.deleted_premise_refused`	PQ	proved
C	RAT proof classified as extension	`PropositionalLRATActions.Controls.rat_refused`	PQ	proved
D	Revalidation is exact	`DerivationGraph.revalidate_iff`	PCQ	proved
D	Admitted extensions transport evidence	`DerivationGraph.replay_extend`	PCQ	proved
D	Checked judgments extend to larger rule systems	`DerivationGraph.CheckedJudgment.extend`	none	proved
D	Retained schedules reinterpret under accepting interfaces	`DerivationGraph.BorrowedCursor.run_reinterpret`	PQ	proved
D	Intuitionistic evidence is classical evidence	`Revalidation.intuitionistic_to_classical`	PCQ	proved
D	Evidence survives appended admitted definitions	`Revalidation.checkDAG_append`	PCQ	proved
D	Negative control: excluded middle refused	`Revalidation.Controls.em_refused_intuitionistically`	P	proved
D	Positive control: extended definitions	`Revalidation.Controls.worked_extended`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Negative control: changed definition	`Revalidation.Controls.worked_changed_refused`	PQ	proved
G	Ten source evidence rules have an exact executable interface	`KripkeTruth.Checking.interface`	P	proved
G	Accepted certificates yield original source evidence	`KripkeTruth.Checking.check_sound`	PQ	proved
G	Certificate coverage is exactly source evidence	`KripkeTruth.Checking.coverage`	PCQ	proved
G	Certificates characterize least-fixed-point membership	`KripkeTruth.Checking.judgment_coverage`	PCQ	proved
G	Closed categorical proofs characterize grounded judgments	`KripkeTruth.Checking.closed_proof_iff_grounded`	PQ	proved
G	Finite decision and certificate authority coincide	`KripkeTruth.Checking.finite_decision_iff_certificate`	PCQ	proved
G	Completed unknown excludes both certificate polarities	`KripkeTruth.Checking.finite_undetermined_iff_no_certificates`	PCQ	proved
G	Positive control: admitted fact	`KripkeTruth.Checking.Controls.fact_accepts`	P	proved
G	Negative control: forged negative conclusion	`KripkeTruth.Checking.Controls.forged_false_refuses`	P	proved
G	Negative control: unsupported truth reference	`KripkeTruth.Checking.Controls.absent_premise_refuses`	P	proved
G	Liar has no certificate at either polarity	`KripkeTruth.Checking.Controls.liar_has_no_certificate`	PCQ	proved
G	Negative control: unadmitted negative base leaf	`KripkeTruth.Checking.Controls.false_base_leaf_refuses`	P	proved
G	Negative control: cyclic proof-graph reference	`KripkeTruth.Checking.Controls.cyclic_graph_refuses`	P	proved
E	Weighted state and branch-cost sequencing is associative	`LayerEffects.Spec.bind_assoc`	PCQ	proved
E	A write and a read need not commute	`LayerEffects.Controls.put_get_not_comm`	PCQ	proved
E	The specified distribution-over-finite-sets law is impossible	`NondeterminismProbability.no_distributiveLaw`	PCQ	proved
E	Surviving answers cannot recover every failed-branch charge	`LayerEffects.Spec.no_failed_cost_recovery`	PCQ	proved
E	Marginals cannot determine same-world conjunction	`EvidenceSpaces.no_same_world_conjunction_from_marginals`	PQ	proved
D	Exact finite local-rule witnesses	`FormationSensitiveChecking.interface`	PQ	proved
D	Original typing iff common tree acceptance	`FormationSensitiveChecking.typing_iff_checked_tree`	PCQ	proved
D	Original typing iff common shared-graph acceptance	`FormationSensitiveChecking.typing_iff_checked_graph`	PCQ	proved
D	Original typing iff logical theorem minting	`FormationSensitiveChecking.typing_iff_minted`	PCQ	proved
D	Maintained replay publishes common authority exactly	`FormationSensitiveChecking.mintReplay_iff`	PQ	proved
D	Full formed judgment iff finite request acceptance	`FormationSensitiveChecking.judgment_iff_checked`	PCQ	proved
D	Admitted substitution preserves checkable source evidence	`FormationSensitiveChecking.checked_substitute`	PCQ	proved
D	Checked request reconstructs the original formed judgment	`FormationSensitiveChecking.Request.sound`	PQ	proved
D	Cumulative tower finite evidence coverage	`Tower.Checking.coverage`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	All positive and negative cumulative controls pass	`Tower.Checking.Controls.all_pass`	PQ	proved
D	Accepted requests hold in the named set-tower model	`TowerInterpretation.Checking.request_model_sound`	PCQ	proved
D	Every closed empty-type request refuses under the model hypothesis	`TowerInterpretation.Checking.empty_request_refuses`	PCQ	proved
D	Checked subjects denote members of their displayed types	`TowerInterpretation.Checking.checked_value_member`	PCQ	proved
D	Checked type conversion preserves actual set values	`TowerInterpretation.Checking.checked_type_conversion`	PCQ	proved
D	Checked subject conversion preserves actual set values	`TowerInterpretation.Checking.checked_term_conversion`	PCQ	proved
D	Checked components construct the original contextual arrow	`TowerInterpretation.Checking.checkedArrow`	PQ	proved
D	Checked values commute with admitted annotated substitution	`TowerInterpretation.Checking.checked_value_substitution`	PCQ	proved
D	Finite closed-universe requests exactly characterize set membership	`TowerInterpretation.Checking.universe_request_iff_set`	PCQ	proved
D	The same requests characterize membership in well-founded hypersets	`TowerInterpretation.Checking.universe_request_iff_hyperset`	PCQ	proved
O	Finite reachability is an equation-invariant native predicate	`GSLTTypeSynthesis.semanticReach`	none	proved
O	Equation-wise route agreement gives equal predicate actions	`EquationRespectingMap.pullback_eq_of_equiv`	PQ	proved
O	Forward strict routes preserve existential next-step observations	`OperationalTranslation.preserves_semanticDiamond`	PQ	proved
O	Operational route agreement preserves semantic reindexing	`OperationalTranslation.pullback_eq_of_equiv`	PQ	proved
O	Local coverage reflects existential next-step observations	`CoveredTranslation.semanticDiamond_iff`	PQ	proved
O	Covered routes preserve and reflect finite reachability	`CoveredTranslation.semanticReach_iff`	PQ	proved
O	Path-valued lowering preserves finite reachability	`OperationalRealization.preserves_semanticReach`	PQ	proved
O	Fusion need not preserve a one-step diamond	`OperationalRealization.FusionCanary.fusion_changes_one_step_diamond`	P	proved
O	Actual covered growth preserves the modal comparison	`IndexedOperational.Canary.covered_growth_semanticDiamond`	PQ	proved
O	Extra target behavior refutes modal reflection	`IndexedOperational.Canary.escaping_native_type_not_reflected`	PQ	proved
O	Outgoing coverage does not preserve the incoming box	`IndexedOperational.Canary.outgoing_coverage_does_not_preserve_past_box`	PQ	proved
O	Conversion paths equal the original structural conversion relation	`FormationSensitiveCheckingOSLF.path_iff_conversion`	P	proved
O	Conversion-native OSLF truth iff accepted finite evidence	`FormationSensitiveCheckingOSLF.satisfies_iff_checked`	PCQ	proved
O	Dependency restriction yields a genuine forward conversion route	`FormationSensitiveCheckingOSLF.restrictionRoute`	PQ	proved
O	Checked higher requests admit a finite sufficient source-reflecting restriction	`FormationSensitiveCheckingOSLF.checked_request_has_restriction`	PQ	proved
O	Reverse beta inhabits the conversion native observation	`Tower.Checking.Controls.reverse_beta_oslf`	PCQ	proved
O	Raw beta observation carries no typing premise	`Tower.Checking.Controls.raw_beta_oslf`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
O	Conversion-native truth does not admit a restricted computation package	`FormationSensitive.RestrictionPreservation.oslf_observation_does_not_admit_restriction`	PCQ	proved
O	The repaired smaller package retains conversion and root preservation	`FormationSensitive.RestrictionPreservation.oslf_observation_with_preserving_repair`	PCQ	proved
O	Guarded OSLF term conversion preserves actual set values	`TowerInterpretation.Checking.oslf_term_conversion`	PCQ	proved
S	Generic conversion observation presents the authored relation	`IndexedOperational.conversionTheory_path_iff`	none	proved
S	Native reachability has exactly the declared conversion support	`IndexedOperational.satisfies_conversionNativeType_iff`	PCQ	proved
S	Simple type interpretation retains the atomic sort	`SingleBaseSTLC.HOLComparison.type_injective`	none	proved
S	Typed HOL syntax retains source terms	`SingleBaseSTLC.HOLComparison.term_injective`	none	proved
S	Renaming commutes with the HOL interpretation	`SingleBaseSTLC.HOLComparison.rename_term`	Q	proved
S	Simultaneous substitution commutes with the HOL interpretation	`SingleBaseSTLC.HOLComparison.substitute_term`	PQ	proved
S	The simple context interpretation is a functor	`SingleBaseSTLC.HOLComparison.contextFunctor`	PQ	proved
S	HOL beta joinability reflects intrinsic beta conversion	`SingleBaseSTLC.HOLComparison.holJoinable_iff_betaConv`	PCQ	proved
S	The HOL and structural tower conversions agree on the image	`SingleBaseSTLC.HOLComparison.holJoinable_iff_towerConv`	PCQ	proved
S	The intrinsic checker decides the HOL observation	`SingleBaseSTLC.HOLComparison.decideConversion_iff_holJoinable`	PCQ	proved
S	Beta conversion supplies retained HOL equality evidence	`SingleBaseSTLC.HOLComparison.equality_proof_of_conversion`	PCQ	proved
S	HOL equality need not reflect into structural conversion	`SingleBaseSTLC.HOLComparison.Controls.hol_equality_does_not_reflect_conversion`	PCQ	proved
S	Typed HOL terms distinguish a raw tower erasure collision	`SingleBaseSTLC.HOLComparison.Controls.hol_retains_erased_types`	none	proved
S	Conversion trees elaborate to retained HOL proofs	`SingleBaseSTLC.HOLConversionBubble.elaborate`	PCQ	proved
S	The beta bubble is exact for structural tower conversion	`SingleBaseSTLC.HOLConversionBubble.beta_path_iff_tower`	PCQ	proved
S	Native predicate truth is the support of retained paths	`SingleBaseSTLC.HOLConversionBubble.satisfies_iff_path`	PCQ	proved
S	The beta native predicate has a total decision procedure	`SingleBaseSTLC.HOLConversionBubble.beta_satisfies_iff_decides`	PCQ	proved
S	Eta strictly enlarges the local conversion policy	`SingleBaseSTLC.HOLConversionBubble.Controls.eta_strictly_adds_conversion`	PCQ	proved
S	The native observation distinguishes beta and eta policies	`SingleBaseSTLC.HOLConversionBubble.Controls.eta_observation_depends_on_policy`	PCQ	proved
S	Pure simple terms require no interpreted constants	`SingleBaseSTLC.HOLReadingComparison.reads`	PQ	proved
S	The actual dependent reader constructs typed codes	`SingleBaseSTLC.HOLReadingComparison.code_typed`	PQ	proved
S	Computed native codes commute with the maintained substitution	`SingleBaseSTLC.HOLReadingComparison.code_substitute`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
S	The beta native observation gives dependent typed equality	`SingleBaseSTLC.HOLReadingComparison.observed_typed`	PCQ	proved
S	Observed equality reaches the explicit set-tower model	`SingleBaseSTLC.HOLTowerModel.observed_set_equality`	PCQ	proved
S	Typed eta equality is distinct from structural conversion	`SingleBaseSTLC.HOLTowerModel.typed_equality_not_structural_conversion`	PCQ	proved
E	Joint admissibility over one base	`extendRules_union_sameDerivability_iff`	P	proved
E	Combine actual certificate compilers	`AdditionalRuleCertificateCompiler.combine`	PCQ	proved
E	Mixed certificate elimination is conservative	`AdditionalRuleCertificateCompiler.combine_sameDerivability`	PCQ	proved
E	Mixed-feature expansion has three base nodes	`ConservativeExtensionCanary.compiledMixed_count`	PCQ	proved
E	Admissible package cannot license a new fact	`ConservativeExtensionCanary.combined_newFact_forbidden`	PCQ	proved
E	Compute retained evidence and recovery receipt	`EqualityRecipe.checkedPathWithRecovery`	PQ	proved
E	Recover the complete submitted recipe	`EqualityRecipe.forget_checkedPath`	PQ	proved
E	Retained evidence replays at its endpoints	`EqualityRecipe.evaluate_forgetPath`	PQ	proved
H	Instantiate licensed equations with scope	`HOL.EquationProofs.instantiate`	PQ	proved
H	Combine equation proof receipts	`HOL.EquationProofs.append`	P	proved
H	Licensed compatible source steps have HOL proofs	`HOL.SourceStep.hasHOLProof`	PCQ	proved
H	Licensed source articles have HOL proofs	`HOL.CoreConversion.hasHOLProof`	PCQ	proved
H	Logical retyping elaborates to HOL evidence	`HOL.ProofSyntaxModulo.hasHOLProof`	PCQ	proved
H	Curry equation cannot obtain HOL receipts	`HOL.ConversionElaborationControls.no_curry_equationProofs`	PCQ	proved
H	Exact local conversion-policy replay	`HOL.ConversionRecipes.leaf_authority`	PCQ	proved
H	Exact authority for selected finite recipes	`HOL.ConversionRecipes.authority`	PCQ	proved
H	Permissions follow the contracted redex type	`HOL.ConversionRecipes.Policy.betaAllowed_mono`	none	proved
H	Policy routes preserve conversion support	`HOL.ConversionRecipes.LocalStep.conversion_mono`	none	proved
H	Construct retained HOL equality proofs	`HOL.ConversionRecipes.elaborate`	PCQ	proved
H	Replace logical casts with equality elimination	`HOL.ConversionRecipes.convertProof`	PCQ	proved
H	Native observation is certificate existence	`HOL.ConversionRecipes.observed_iff_accepted`	PCQ	proved
H	Operational observation follows policy edges	`HOL.ConversionRecipes.observed_mono`	PCQ	proved
H	Untrusted proposals produce checked HOL evidence	`HOL.ConversionRecipes.elaborateSubmission`	PCQ	proved
H	Convergent equality retains different receipts	`HOL.ConversionRecipes.Controls.eta_and_equation_recipes_distinct`	none	proved
H	Equal endpoints retain different proof sizes	`HOL.ConversionRecipes.Controls.proof_costs_differ`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
H	Two-sort conversion and elaboration controls	`HOL.ConversionRecipes.Controls.all_verdicts`	PCQ	proved
H	Least upper policy for joined generators	`HOL.ConversionRecipes.Policy.join_extends_iff`	P	proved
H	Compile actual typed beta executions	`HOL.ConversionRecipes.executionRecipe`	PCQ	proved
H	The execution compiler produces accepted recipes	`HOL.ConversionRecipes.executionRecipe_checked`	PCQ	proved
H	Previous beta observation reaches the recipe checker	`HOL.ConversionRecipes.joinable_has_betaRecipe`	PCQ	proved
I	Retain selected schema positions and substitutions	`SchemaTableConversionCode.retained`	PQ	proved
I	Finite schema decoding is sound	`SchemaTableConversionCode.decode_sound`	PQ	proved
I	Finite schema decoding covers every selected instance	`SchemaTableConversionCode.decode_complete`	PQ	proved
I	Qualify selected schema-root replay	`SchemaTableConversionCode.rootDecoder`	PQ	proved
I	Root replay commutes with renaming	`SchemaTableConversionCode.decode_rename`	PQ	proved
I	Root replay commutes with substitution	`SchemaTableConversionCode.decode_substitute`	PQ	proved
I	Check formed dependent substitutions	`FormationSensitiveChecking.SubstitutionRequest.sound`	PQ	proved
I	Admitted substitutions have finite evidence	`FormationSensitiveChecking.SubstitutionRequest.complete`	PCQ	proved
I	Compute the original typed schema receipt	`IndexedFamily.Checking.receiptAt`	PQ	proved
I	Reconstruct the original iota witness	`IndexedFamily.Checking.realizeEvidence`	PQ	proved
I	Selected computation includes into the declaration calculus	`IndexedFamily.Checking.inclusion`	PQ	proved
I	Inclusion retains full formed judgments	`IndexedFamily.Checking.judgment_inclusion`	PQ	proved
I	Selected conversion has a forward operational route	`IndexedFamily.Checking.conversionRoute`	PQ	proved
I	Checked instantiation reconstructs typed iota evidence	`IndexedFamily.Checking.InstanceRequest.replay`	PQ	proved
I	Typed iota publication checks its target context	`IndexedFamily.Checking.InstanceRequest.formed`	PQ	proved
I	Typed requests export accepted root codes	`IndexedFamily.Checking.InstanceRequest.rootCode_accepted`	PQ	proved
I	Exact decoding of all four Nat/Vec iota cases	`NativeNaturalVectorFamilies.Checking.iotaDecoder`	PCQ	proved
I	Join actual Nat/Vec roots with declaration replay	`NativeNaturalVectorFamilies.Checking.decoder`	PCQ	proved
I	Actual Nat/Vec conversion has exact finite evidence	`NativeNaturalVectorFamilies.Checking.conversion_iff_checked`	PCQ	proved
I	Connect the Nat/Vec formation-sensitive checker	`NativeNaturalVectorFamilies.Checking.coverage`	PCQ	proved
I	Typed iota and dependent-substitution controls	`IndexedFamilyCheckingControls.all_verdicts`	PCQ	proved
E	Replay a generated row at its original ordinal	`SchemaTableConversionCode.decode_at`	PQ	proved
E	All selected schema roots equal the original schema support	`SchemaTableConversionCode.all_roots_iff`	PQ	proved
E	Use independent source presentations in common replay	`SchemaTableConversionCode.presentedDecoder`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
E	Generated simple iota rules have the exact finite presentation	`IndexedPolynomialGenerated.Checking.presents`	PQ	proved
E	Connect generated roots to the common conversion decoder	`IndexedPolynomialGenerated.Checking.decoder`	PQ	proved
E	Recover the schema of the original constructor occurrence	`IndexedPolynomialGenerated.Checking.table_get`	PQ	proved
E	Replay generated receipts with their complete assignments	`IndexedPolynomialGenerated.Checking.receiptCode_accepted`	PQ	proved
E	Generated conversion has exact finite evidence	`IndexedPolynomialGenerated.Checking.conversion_iff_checked`	PQ	proved
E	Generated receipts compile to accepted recipes	`IndexedPolynomialGenerated.Checking.receiptRecipe_accepted`	PQ	proved
E	Accepted submissions give original formed judgments	`FormationSensitiveElaboration.replay_sound`	PQ	proved
E	Bidirectional evidence production is sound after replay	`FormationSensitiveElaboration.elaborate_sound`	PQ	proved
E	Inferred implicit assignments retain all determined slots	`FormationSensitiveElaboration.Implicit.complete_sound`	PQ	proved
E	Implicit insertion checks the actual rendered application	`FormationSensitiveElaboration.Implicit.elaborate_sound`	PQ	proved
E	Generated replay and elaboration controls	`InductiveElaborationControls.all_verdicts`	PCQ	proved
E	Infer both omitted vector-cons arguments and check the result	`InductiveElaborationControls.inferredSingleton_typed`	PCQ	proved
E	Proposed normalization carries a checked conversion recipe	`StructuralConversionSearch.normalize_sound`	PQ	proved
E	Bounded comparison produces sound conversion evidence	`StructuralConversionSearch.compare_sound`	PQ	proved
E	Instantiate the motive universe at the element/motive maximum	`NativeNaturalVectorFamilies.Programs.levelInstance`	PQ	proved
E	The selected Nat/Vec universe instance is formed	`NativeNaturalVectorFamilies.Programs.signature_formed`	PCQ	proved
E	Transport full judgments through that universe instance	`NativeNaturalVectorFamilies.Programs.judgment_map`	PQ	proved
E	Independent replay accepts the finite append evidence	`NativeNaturalVectorFamilies.Programs.appendChecked_true`	PCQ	proved
E	The complete closed dependent append function is typed	`NativeNaturalVectorFamilies.Programs.append_typed`	PCQ	proved
E	Refuse changed append result indices in the submitted evidence	`NativeNaturalVectorFamilies.Programs.changedType_refused`	PCQ	proved
E	Refuse a changed append program with the same submission	`NativeNaturalVectorFamilies.Programs.changedTerm_refused`	PCQ	proved
E	Refuse a forged append root typing rule	`NativeNaturalVectorFamilies.Programs.forgedRoot_refused`	PCQ	proved
E	Dependent append acceptance and refusal controls	`NativeNaturalVectorFamilies.Programs.all_verdicts`	PCQ	proved
I	Compute a retained natural-eliminator receipt	`IndexedFamilyCheckingControls.checkedZeroReceipt`	PCQ	proved
K	Append commutes with capture-safe substitution	`NativeNaturalVectorFamilies.Programs.Computation.append_substitute`	PQ	proved
K	Append nil conversion holds for arbitrary scoped arguments	`NativeNaturalVectorFamilies.Programs.Computation.append_nil`	PCQ	proved
K	Append cons conversion retains the recursive append result	`NativeNaturalVectorFamilies.Programs.Computation.append_cons`	PCQ	proved
K	Refuse omission of append recursive computation	`NativeNaturalVectorFamilies.Programs.Computation.wrongBranch_refused`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
K	Symbolic append computation controls	`NativeNaturalVectorFamilies.Programs.Computation.all_verdicts`	PCQ	proved
K	Inverse substitution reconstructs its original scoped source	`ScopedSubstitutionInversion.invert_sound`	PQ	proved
K	Recursive-call abstraction preserves binder scope	`ScopedSubstitutionInversion.Controls.all_verdicts`	PQ	proved
K	Constructor clauses retain their actual names and arities	`IndexedInductive.CaseElaboration.adaptClause_name`	none	proved
K	Compiled methods reconstruct written recursive child calls	`IndexedInductive.CaseElaboration.compileBranch_reconstructs`	PQ	proved
K	Constructor clauses cover the exact ordered admitted inventory	`IndexedInductive.CaseElaboration.compileMethods_coverage`	PQ	proved
K	Constructor-case elaboration checks its actual eliminator application	`IndexedInductive.CaseElaboration.elaborate_sound`	PQ	proved
K	Written append clauses generate the independent reference methods	`IndexedInductive.AppendElaboration.methods_match_reference`	PQ	proved
K	Scoped append compilation gives the checked closed program	`IndexedInductive.AppendElaboration.compiled_matches_reference`	PQ	proved
K	The compiled append function retains its dependent type	`IndexedInductive.AppendElaboration.compiled_typed`	PCQ	proved
K	The source-compiled append is typed by the natural-only policy	`IndexedInductive.AppendElaboration.compiled_typed_naturalOnly`	PCQ	proved
K	Uniform scoped append clause acceptance and refusal controls	`IndexedInductive.AppendElaboration.all_verdicts`	PQ	proved
K	Retain schema evidence when permissions increase	`SchemaTableConversionCode.Evidence.weakenPermissions`	P	proved
K	Schema-root permissions include monotonically	`SchemaTableConversionCode.root_mono`	PQ	proved
K	Restricted Nat/Vec computation includes into the original signature	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.includes_original`	PCQ	proved
K	Nat/Vec declaration permissions include monotonically	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.permissions_mono`	PCQ	proved
K	Restricted checking preserves full original judgments	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.judgment_inclusion`	PCQ	proved
K	Existing append evidence checks with the two natural iota rows	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.append_checked_naturalOnly`	PCQ	proved
K	The smaller policy types the full dependent append function	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.append_typed_naturalOnly`	PCQ	proved
K	Checking and computation permission controls	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.all_verdicts`	PCQ	proved
K	The admitted natural eliminator computes at zero	`IndexedInductiveNaturalHOL.recurse_zero`	P	proved
K	The admitted natural eliminator computes at successor	`IndexedInductiveNaturalHOL.recurse_succ`	P	proved
K	The natural recursion equations determine their actual function	`IndexedInductiveNaturalHOL.recurse_unique`	PQ	proved
K	The authored natural carrier has unique recursion	`IndexedInductiveNaturalHOL.recursion_exists_unique`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
K	The authored natural carrier has full predicate induction	`IndexedInductiveNaturalHOL.induction`	PQ	proved
K	The authored carrier supplies the existing Peano structure	`IndexedInductiveNaturalHOL.peanoStructure`	PQ	proved
K	The source-carrier HOL interpretation has full domains	`IndexedInductiveNaturalHOL.fullDomains`	PQ	proved
K	The source carrier satisfies the existing HOL Peano formulas	`IndexedInductiveNaturalHOL.models_axioms`	PQ	proved
K	The HOL interpretation retains natural no-confusion	`IndexedInductiveNaturalHOL.models_noConfusion`	PQ	proved
K	The HOL interpretation satisfies second-order induction	`IndexedInductiveNaturalHOL.models_induction`	PQ	proved
K	The source eliminator satisfies the HOL unique-recursion formula schema	`IndexedInductiveNaturalHOL.models_recursion`	PQ	proved
K	The authored natural carrier is infinite	`IndexedInductiveNaturalHOL.infinite_carrier`	PCQ	proved
K	The actual interpreted successor cannot collapse to zero	`IndexedInductiveNaturalHOL.successor_zero_refused`	PQ	proved
K	Finite case plans reconstruct the written recursive calls	`IndexedInductive.CaseElaboration.compilePlan_reconstructs`	PQ	proved
K	Finite case plans retain exact ordered clause coverage	`IndexedInductive.CaseElaboration.compilePlans_coverage`	PQ	proved
K	The selected direct module reconstructs every source block	`IndexedInductive.OperationSource.selectModule_exact`	P	proved
K	Lexical symbol expansion commutes with substitution	`IndexedInductive.OperationSource.Symbol.instantiate_substitute`	PQ	proved
K	Named symbol applications retain substituted arguments	`IndexedInductive.OperationSource.applySymbol_substitute`	PQ	proved
K	Named operation lowering commutes with substitution	`IndexedInductive.OperationSource.lower_substitute`	PQ	proved
K	Named operation expansion and scope controls	`IndexedInductive.OperationSource.Controls.all_verdicts`	PCQ	proved
K	Named constructor-clause and resolution controls	`IndexedInductive.OperationSource.Controls.all_clauseVerdicts`	PCQ	proved
K	The actual textual program retains its complete module data	`IndexedInductive.AppendSource.source_reconstructed`	P	proved
K	Textual declarations lower to the independent admitted vectors	`IndexedInductive.AppendSource.text_admitted`	P	proved
K	Executable source plans agree with the admitted semantic inventory	`IndexedInductive.AppendSource.source_vector_plans_match`	PCQ	proved
K	Textual addition compiles from its own source clauses	`IndexedInductive.AppendSource.addition_matches_reference`	PCQ	proved
K	Textual append compiles to the independent checked core	`IndexedInductive.AppendSource.append_matches_reference`	PCQ	proved
K	The actual textual append has the full dependent function type	`IndexedInductive.AppendSource.append_typed`	PCQ	proved
K	Textual append retains its type in the natural-only policy	`IndexedInductive.AppendSource.append_typed_naturalOnly`	PCQ	proved
K	Independent replay checks the actual textual producer output	`IndexedInductive.AppendSource.checkProposal_sound`	PCQ	proved
K	Textual append acceptance and malformed-source controls	`IndexedInductive.AppendSource.all_verdicts`	PCQ	proved
L	The exact closure body denotes quantified predicate closure	`HOL.NaturalClosure.denote_body`	none	proved
L	Applied native closure retains its selected model meaning	`HOL.NaturalClosure.denote_predicate`	none	proved

Part	Claim	Lean declaration	Axioms	Status
L	Every base point satisfies its closure predicate	`HOL.NaturalClosure.closedUnder_zero`	none	proved
L	Admitted successors preserve predicate closure	`HOL.NaturalClosure.closedUnder_successor`	none	proved
L	Strengthened closure gives induction in full-domain models	`HOL.NaturalClosure.closedUnder_induction`	none	proved
L	The exact zero statement is valid in the selected Henkin semantics	`HOL.NaturalClosure.models_zero`	none	proved
L	The exact successor statement is valid in Henkin semantics	`HOL.NaturalClosure.models_successor`	none	proved
L	The exact induction statement holds in full-domain models	`HOL.NaturalClosure.models_induction`	none	proved
N4	Binder naming commutes with lifted variable renaming	`HOL.Names.lift_comp`	Q	proved
N4	Intrinsic HOL renaming commutes with MM0 HOL syntax production	`MM0.HOLSyntaxWire.encodeTerm_rename`	PQ	proved
N4	Printed natural-closure body retains the shared Henkin meaning	`MM0.NaturalClosure.definition_meaning`	none	proved
N4	Concrete MM0 closure names retain binder separation	`MM0.NaturalClosure.reference_names_distinct`	none	proved
N4	Zero is generated in the finite collapsing interpretation	`MM0.NaturalClosure.finite_zero`	none	proved
N4	Another finite inhabitant need not belong to natural closure	`MM0.NaturalClosure.finite_other_refused`	none	proved
N4	MM0 HOL closure reference controls pass	`MM0.NaturalClosure.all_verdicts`	P	proved
L	Independent intrinsic HOL syntax erases to the native closure body	`Megalodon.NaturalClosure.body_erases`	P	proved
L	The actual native closure definition has its declared type	`Megalodon.NaturalClosure.native_body_typed`	P	proved
L	The three native closure statements are formed	`Megalodon.NaturalClosure.native_statements_formed`	P	proved
L	Native zero unfolding agrees with the intrinsic statement	`Megalodon.NaturalClosure.zero_statement_erases`	PCQ	proved
L	Native successor unfolding agrees with the intrinsic statement	`Megalodon.NaturalClosure.successor_statement_erases`	PCQ	proved
L	Native induction unfolding agrees with the intrinsic statement	`Megalodon.NaturalClosure.induction_statement_erases`	PCQ	proved
L	The finite native zero proof checks against its exact claim	`Megalodon.NaturalClosure.zero_proof_checked`	PCQ	proved
L	The finite native successor proof checks against its exact claim	`Megalodon.NaturalClosure.successor_proof_checked`	PCQ	proved
L	Native closure acceptance and altered-evidence controls	`Megalodon.NaturalClosure.all_verdicts`	PCQ	proved
L	Every authored natural satisfies the exact closure predicate	`IndexedInductiveNaturalMegalodon.source_member`	PQ	proved
L	The source interpretation satisfies the closure membership formula	`IndexedInductiveNaturalMegalodon.source_membership_formula`	PQ	proved
L	The three native formulas hold on the authored carrier	`IndexedInductiveNaturalMegalodon.source_native_formulas`	PQ	proved
L	Closure-restricted induction retains every authored source value	`IndexedInductiveNaturalMegalodon.source_induction`	PQ	proved
L	A collapsed successor excludes a genuine source successor	`IndexedInductiveNaturalMegalodon.collapsing_successor_refused`	PQ	proved
L	The collapsed-operation negative has nonempty closure	`IndexedInductiveNaturalMegalodon.collapsing_zero_accepted`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
L	Composed certificate links agree with their address composition	`SchemaTableConversionCode.Linking.relocate_comp`	P	proved
L	Schema relocation preserves both acceptance and refusal	`SchemaTableConversionCode.Linking.decode_relocate`	PQ	proved
L	A schema embedding preserves the selected root judgment	`SchemaTableConversionCode.Linking.root_inclusion`	PQ	proved
L	Package linking and permission controls	`SchemaTableConversionCode.Linking.Controls.all_verdicts`	PQ	proved
L	The informative source vector table joins the exact natural prefix	`IndexedInductive.AppendChecking.generated_table_reference`	PCQ	proved
L	The generated library matches the independent declaration inventory	`IndexedInductive.AppendChecking.generated_library_reference`	PCQ	proved
L	The informative candidate retains the reference declaration entries	`IndexedInductive.AppendChecking.candidate_entries_reference`	PCQ	proved
L	Candidate computation includes into the independent Nat/Vec roots	`IndexedInductive.AppendChecking.candidate_root_inclusion`	PCQ	proved
L	Both declarations and roots extend from the generated candidate	`IndexedInductive.AppendChecking.candidate_extension`	PCQ	proved
L	The generated source calculus preserves full reference judgments	`IndexedInductive.AppendChecking.candidate_judgment_inclusion`	PCQ	proved
L	The candidate route composes with the selected universe instance	`IndexedInductive.AppendChecking.candidate_judgment_to_program`	PCQ	proved
L	Generated and executable checking policies agree exactly	`IndexedInductive.AppendChecking.signature_comparison`	PCQ	proved
L	The actual textual declarations select the informative candidate table	`IndexedInductive.AppendChecking.textual_candidate_table`	PCQ	proved
L	The actual textual append is typed by the generated checking policy	`IndexedInductive.AppendChecking.textual_append_typed`	PCQ	proved
L	Finite replay checks the term in the generated package policy	`IndexedInductive.AppendChecking.replay_sound`	PCQ	proved
L	Vector receipts retain their source bounds and global permissions	`IndexedInductive.AppendChecking.linked_vector_replay`	PCQ	proved
L	Package linking and permission controls	`IndexedInductive.AppendChecking.all_verdicts`	PCQ	proved
L	The natural-only package policy checks the actual textual append	`IndexedInductive.AppendChecking.replay_naturalOnly`	PCQ	proved
L	Fixed-row typed substitutions have exact finite certificate coverage	`Declaration.IndexedFamily.Checking.InstanceRequest.checked_arguments_iff`	PCQ	proved
L	Selected original rows present exactly the selected roots	`SchemaTableConversionCode.selected_roots_iff`	PQ	proved
L	Opaque declaration extensions expose only their declared roots	`DeclarationConversionCode.opaque_roots_iff`	PQ	proved
L	The finite pattern check establishes the existing lifting condition	`Annotated.SchemaTable.linearCheck_sound`	PQ	proved
L	Linear selected patterns lift every raw root at an annotated input	`Annotated.SchemaTable.root_lift`	PQ	proved
L	Root permissions retain the annotated declarations	`Programs.Annotated.constantType_eq`	PCQ	proved
L	The actual natural-only roots lift uniformly with annotations	`Programs.Annotated.lift_natural_root`	PCQ	proved
L	The vector nil pattern repeats its parameter	`Programs.Annotated.vector_nil_repeats_parameter`	P	proved
L	The vector cons pattern repeats its length index	`Programs.Annotated.vector_cons_repeats_index`	P	proved
L	All four canonical endpoints have independent annotated typing	`Programs.Annotated.endpoints_typed`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
L	The actual annotated canonical telescopes are formed	`Programs.Annotated.context_formed`	PCQ	proved
L	Canonical right sides typecheck against left-inferred templates	`Programs.Annotated.template_typed`	PCQ	proved
L	Annotated root preservation conditional on former injectivity	`Programs.Annotated.root_preserving_of_formerFacts`	PCQ	proved
L	Former inversion discharges the actual schema premises	`Programs.Annotated.root_premised_of_formerFacts`	PCQ	proved
L	Former facts suffice for the actual annotated root equalities	`Programs.Annotated.root_admitted_of_formerFacts`	PCQ	proved
L	Licensed rows give typed equalities under typed assignments	`Programs.Annotated.instantiated_equal`	PCQ	proved
L	Erasure preserves the selected typed instance equality	`Programs.Annotated.erased_instantiated_equal`	PCQ	proved
L	Each original row has a canonical typed equality witness	`Programs.Annotated.canonical_equal`	PCQ	proved
L	Canonical typing metadata and changed-type refusal controls	`Programs.Annotated.all_metadataVerdicts`	PCQ	proved
L	Natural lifting and repeated vector-parameter controls	`Programs.Annotated.all_verdicts`	PCQ	proved
L	The full textual declaration source supplies the complete generated library	`IndexedInductive.AppendChecking.textual_candidate_library`	P	proved
L	Actual submitted proposals replay in the generated policy	`IndexedInductive.AppendChecking.replayProposal_sound`	PCQ	proved
L	Live source proposals retain their generated checking policy	`IndexedInductive.AppendChecking.replaySource_sound`	PCQ	proved
I	The recursion graph is functional under successor separation	`NaturalRecursion.Graph.functional`	none	proved
I	Graph values agree with every recursive solution	`NaturalRecursion.Graph.agrees`	none	proved
I	Every generated number has a graph value	`NaturalRecursion.Graph.exists_of_generated`	none	proved
I	A lawful selector selects a graph value	`NaturalRecursion.selected_graph`	none	proved
I	The selected recursor satisfies its zero equation	`NaturalRecursion.selected_zero`	none	proved
I	The selected recursor satisfies its successor equation	`NaturalRecursion.selected_successor`	none	proved
I	Selected recursion is unique on the generated carrier	`NaturalRecursion.selected_unique`	none	proved
I	Full-domain Leibniz equality has the declared meaning	`NaturalRecursion.denote_equality`	none	proved
I	Full-domain encoded existence has the declared meaning	`NaturalRecursion.denote_existential`	none	proved
I	The typed graph denotes the independent inductive graph	`NaturalRecursion.denote_graph_iff`	none	proved
I	The typed uniqueness clause has its pointwise meaning	`NaturalRecursion.denote_uniqueness`	PQ	proved
I	The typed zero-separation interface retains its meaning	`NaturalRecursion.denote_zeroExcluded`	PQ	proved
I	The typed injectivity interface retains its meaning	`NaturalRecursion.denote_successorInjective`	PQ	proved
I	The typed choice interface retains its selection law	`NaturalRecursion.denote_choiceCondition`	PQ	proved
I	The typed recursion clauses retain their equations	`NaturalRecursion.denote_recursionProperty`	PQ	proved
I	Full-domain models satisfy conditional unique recursion	`NaturalRecursion.models_recursion`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	The independent typed graph matches its exported definition	`NaturalRecursion.graph_source_definition`	P	proved
I	Typed Leibniz equality matches the exported definition	`NaturalRecursion.equality_source_definition`	P	proved
I	Typed existence matches the exported definition	`NaturalRecursion.existential_source_definition`	P	proved
I	The independent typed recursion statement erases exactly	`NaturalRecursion.native_statement_erases`	P	proved
I	The independent native recursion statement is formed	`NaturalRecursion.native_statement_formed`	P	proved
I	The authored eliminator supplies a recursion graph value	`IndexedInductiveNaturalMegalodon.source_recursion_graph`	PQ	proved
I	The graph is exactly the authored eliminator graph	`IndexedInductiveNaturalMegalodon.source_graph_iff`	PQ	proved
I	Every lawful selector agrees with the authored eliminator	`IndexedInductiveNaturalMegalodon.source_selector_agrees`	PQ	proved
I	Lawful selectors agree on every authored natural	`IndexedInductiveNaturalMegalodon.source_selectors_agree`	PQ	proved
I	The classical interpretation supplies scoped inhabited choice	`IndexedInductiveNaturalMegalodon.source_recursion_with_choice`	PCQ	proved
I	The authored full-domain model satisfies the native schema	`IndexedInductiveNaturalMegalodon.source_recursion_statement`	PQ	proved
I	Source successor discharges both typed interfaces	`IndexedInductiveNaturalMegalodon.source_successor_formulas`	PQ	proved
I	The recursion graph contains a genuine successor result	`IndexedInductiveNaturalMegalodon.source_recursion_nonempty`	none	proved
I	A constant-zero selector fails a genuine source computation	`IndexedInductiveNaturalMegalodon.constant_selector_wrong_result`	PQ	proved
I	Original selected natural rows are exactly the constructor schemas	`ComputationProfiles.NaturalConversion.schema_iff`	PCQ	proved
I	Actual natural-only roots have exact schema coverage	`ComputationProfiles.NaturalConversion.roots_exact`	PCQ	proved
I	The actual natural schemas form a constructor system	`ComputationProfiles.NaturalConversion.system`	PCQ	proved
I	Natural-only structural conversion is confluent	`ComputationProfiles.NaturalConversion.churchRosser`	PCQ	proved
I	Natural-only conversion has the Pi boundary	`ComputationProfiles.NaturalConversion.pi_boundary`	PCQ	proved
I	Natural-only conversion has the Sigma boundary	`ComputationProfiles.NaturalConversion.sigma_boundary`	PCQ	proved
I	Natural-only roots require the original four-argument eliminator	`ComputationProfiles.NaturalConversion.root_head`	PCQ	proved
I	Bare declared constants do not compute in this policy	`ComputationProfiles.NaturalConversion.constant_normal`	PCQ	proved
I	Actual addition at zero is a permitted conversion	`ComputationProfiles.NaturalConversion.addition_zero`	PCQ	proved
I	Structural conversion separates the natural type from Pi	`ComputationProfiles.NaturalConversion.natural_not_pi`	PCQ	proved
I	Typed substitutions construct arbitrary dependent declaration spines	`FormationSensitive.DeclarationSpine.applyTelescope`	PQ	proved
I	Observed recursor typing retains its genuine declaration formation	`ComputationProfiles.NaturalConversion.Preservation.recursor_spine`	PCQ	proved
I	Recover actual dependent recursor arguments and result adjustment	`ComputationProfiles.NaturalConversion.Preservation.recover`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	The actual selected zero root uniformly preserves typing	`ComputationProfiles.NaturalConversion.Preservation.zero_preserves`	PCQ	proved
I	Recover a successor predecessor at its declared domain	`ComputationProfiles.NaturalConversion.Preservation.successor_argument`	PCQ	proved
I	The actual selected successor root uniformly preserves typing	`ComputationProfiles.NaturalConversion.Preservation.successor_preserves`	PCQ	proved
I	All actual natural-only roots uniformly preserve typing	`ComputationProfiles.NaturalConversion.Preservation.roots`	PCQ	proved
I	Natural-only contextual computation preserves full formed judgments	`ComputationProfiles.NaturalConversion.Preservation.steps_preserve`	PCQ	proved
I	An actual checked zero redex witnesses preservation	`ComputationProfiles.NaturalConversion.Controls.zero_typed`	PCQ	proved
I	An actual checked successor redex witnesses preservation	`ComputationProfiles.NaturalConversion.Controls.successor_typed`	PCQ	proved
I	The uniform zero theorem applies to a real checked redex	`ComputationProfiles.NaturalConversion.Controls.zero_preserved`	PCQ	proved
I	Four actual steps compute the selected successor example	`ComputationProfiles.NaturalConversion.Controls.successor_path`	PCQ	proved
I	The full successor computation retains its exact type	`ComputationProfiles.NaturalConversion.Controls.successor_preserved`	PCQ	proved
I	The same natural proof submission refuses an altered Pi result	`ComputationProfiles.NaturalConversion.Controls.altered_type_refused`	PCQ	proved
I	Natural preservation controls accept and refuse their intended inputs	`ComputationProfiles.NaturalConversion.Controls.all_verdicts`	PCQ	proved
N5	The original Nat declaration retains its universe typing	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.natural_typed`	PCQ	proved
N5	The original zero declaration has type Nat	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.zero_typed`	PCQ	proved
N5	The original successor has its dependent-function type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.successor_constant_typed`	PCQ	proved
N5	Successor preserves the actual annotated Nat type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.successor_typed`	PCQ	proved
N5	Successor transports actual typed natural equalities	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.successor_equal`	PCQ	proved
N5	The natural type lies in the selected motive universe	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.natural_in_universe`	PCQ	proved
N5	The selected motive universe has its successor typing	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.universe_typed`	PCQ	proved
N5	The natural motive function type is formed	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.motive_type`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
N5	The actual annotated natural motive has its declared type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.motive_typed`	PCQ	proved
N5	The applied actual motive retains its universe typing	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.motive_applied`	PCQ	proved
N5	The actual motive computes by a finite typed beta equality	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.motive_beta`	PCQ	proved
N5	The recursive natural-method domain is formed	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_inner_type`	PCQ	proved
N5	The full natural method function type is formed	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_type`	PCQ	proved
N5	The actual method body retains its indexed motive result	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.method_body_typed`	PCQ	proved
N5	The actual recursive method abstraction is typed	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_inner_typed`	PCQ	proved
N5	The complete annotated natural method has its declared type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_typed`	PCQ	proved
N5	Two typed beta steps compute the actual natural method	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_apply_equal`	PCQ	proved
N5	The original natural recursor has its closed telescope type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.recursor_typed`	PCQ	proved
N5	Actual recursor arguments form a typed simultaneous substitution	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.arguments_typed`	PCQ	proved
N5	Recursor application retains the actual indexed motive result	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.eliminate_typed`	PCQ	proved
N5	Annotated addition is typed for arbitrary typed natural arguments	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_typed`	PCQ	proved
N5	Annotated addition erases to the independently defined M1 code	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_erases`	P	proved
N5	The complete annotated addition function has its curried type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.addFunction_typed`	PCQ	proved
N5	The closed annotated function erases to actual closed M1 addition	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.addFunction_erases`	P	proved
N5	The existing bidirectional annotator reconstructs typed addition	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.producedAddFunction_exact`	PCQ	proved
N5	The actual produced addition has an independent typing derivation	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.producedAddFunction_typed`	PCQ	proved
N5	The actual typed zero equation retains original row permission	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_zero`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
N5	The actual typed successor equation retains original row permission	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_successor`	PCQ	proved
N5	The inactive annotated profile has no root computation	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_no_root`	PCQ	proved
N5	Closed addition stays typed with every iota root disabled	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.typed_without_iota`	PCQ	proved
N5	A second actual motive has its own genuine domain typing	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.otherMotive_typed`	PCQ	proved
N5	Different typed motive domains share one erasure	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.domain_erasure_distinction`	none	proved
N5	Actual annotated one plus zero equals one at Nat	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.one_plus_zero`	PCQ	proved
N5	Annotated natural-program reference controls pass	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.all_verdicts`	PCQ	proved
I	Actual annotated addition commutes with substitution	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_substitute`	P	proved
I	Actual annotated addition commutes with renaming	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_rename`	P	proved
I	Abstract an independently typed body over formed dependent domain and codomain	`TypedEquality.Annotated.CTyped.abstractTerm`	P	proved
I	Actual dependent append: elementSort typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.elementSort_typed`	PCQ	proved
I	Actual dependent append: in motive universe	`NativeNaturalVectorFamilies.Programs.Annotated.Append.in_motive_universe`	PCQ	proved
I	Actual dependent append: vector constant typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.vector_constant_typed`	PCQ	proved
I	Actual dependent append: vector typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.vector_typed`	PCQ	proved
I	Actual dependent append: vector index equal	`NativeNaturalVectorFamilies.Programs.Annotated.Append.vector_index_equal`	PCQ	proved
I	Actual dependent append: nil constant typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_constant_typed`	PCQ	proved
I	Actual dependent append: nil typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_typed`	PCQ	proved
I	Actual dependent append: cons constant typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.cons_constant_typed`	PCQ	proved
I	Actual dependent append: cons typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.cons_typed`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Actual dependent append: result typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.result_typed`	PCQ	proved
I	Actual dependent append: result substitute	`NativeNaturalVectorFamilies.Programs.Annotated.Append.result_substitute`	P	proved
I	Actual dependent append: innerType open	`NativeNaturalVectorFamilies.Programs.Annotated.Append.innerType_open`	PQ	proved
I	Actual dependent append: family applied	`NativeNaturalVectorFamilies.Programs.Annotated.Append.family_applied`	PCQ	proved
I	Actual dependent append: recursor constant typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.recursor_constant_typed`	PCQ	proved
I	Actual dependent append: arguments typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.arguments_typed`	PCQ	proved
I	Actual dependent append: eliminate typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.eliminate_typed`	PCQ	proved
I	Actual dependent append: innerType typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.innerType_typed`	PCQ	proved
I	Actual dependent append: familyType typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.familyType_typed`	PCQ	proved
I	Actual dependent append: motiveInner typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motiveInner_typed`	PCQ	proved
I	Actual dependent append: motive typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motive_typed`	PCQ	proved
I	Actual dependent append: motiveInner substitute	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motiveInner_substitute`	P	proved
I	Actual dependent append: motiveInner rename	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motiveInner_rename`	P	proved
I	Actual dependent append: motive rename	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motive_rename`	P	proved
I	Actual dependent append: motive beta	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motive_beta`	PCQ	proved
I	Actual dependent append: method body typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.method_body_typed`	PCQ	proved
I	Actual dependent append: method typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.method_typed`	PCQ	proved
I	Actual dependent append: nil branch typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_branch_typed`	PCQ	proved
I	Actual dependent append: append typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_typed`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Actual dependent append: functionContext formed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.functionContext_formed`	PCQ	proved
I	Actual dependent append: functionBody typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.functionBody_typed`	PCQ	proved
I	Actual dependent append: functionResult typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.functionResult_typed`	PCQ	proved
I	Actual dependent append: appendFunction typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunction_typed`	PCQ	proved
I	Actual dependent append: motive erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motive_erases`	PQ	proved
I	Actual dependent append: method erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.method_erases`	PQ	proved
I	Actual dependent append: append erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_erases`	PQ	proved
I	Actual dependent append: appendFunction erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunction_erases`	PQ	proved
I	Actual dependent append: appendFunctionType erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunctionType_erases`	P	proved
I	Actual dependent append: producedAppendFunction exact	`NativeNaturalVectorFamilies.Programs.Annotated.Append.producedAppendFunction_exact`	PCQ	proved
I	Actual dependent append: producedAppendFunction typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.producedAppendFunction_typed`	PCQ	proved
I	Actual dependent append: naturalOnly typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.naturalOnly_typed`	PCQ	proved
I	Actual dependent append: unconstrained erasure distinction	`NativeNaturalVectorFamilies.Programs.Annotated.Append.unconstrained_erasure_distinction`	PCQ	proved
I	Actual dependent append: all verdicts	`NativeNaturalVectorFamilies.Programs.Annotated.Append.all_verdicts`	PCQ	proved
I	Typed append computation: methodContext formed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.methodContext_formed`	PCQ	proved
I	Typed append computation: methodResult typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.methodResult_typed`	PCQ	proved
I	Typed append computation: method apply equal	`NativeNaturalVectorFamilies.Programs.Annotated.Append.method_apply_equal`	PCQ	proved
I	Typed append computation: append nil	`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_nil`	PCQ	proved
I	Typed append computation: append cons	`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_cons`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Typed append computation: natural as element	`NativeNaturalVectorFamilies.Programs.Annotated.Append.natural_as_element`	PCQ	proved
I	Typed append computation: closed nil	`NativeNaturalVectorFamilies.Programs.Annotated.Append.closed_nil`	PCQ	proved
I	Typed append computation: closed cons	`NativeNaturalVectorFamilies.Programs.Annotated.Append.closed_cons`	PCQ	proved
I	Typed append computation: closed cons erased	`NativeNaturalVectorFamilies.Programs.Annotated.Append.closed_cons_erased`	PCQ	proved
I	Typed append computation: typed with vector rows disabled	`NativeNaturalVectorFamilies.Programs.Annotated.Append.typed_with_vector_rows_disabled`	PCQ	proved
I	Typed append computation: all computationVerdicts	`NativeNaturalVectorFamilies.Programs.Annotated.Append.all_computationVerdicts`	PCQ	proved
I	Restrict a typed telescope assignment to its ambient context	`TypedEquality.Annotated.CTele.ambientSub_typed`	P	proved
I	Opening after ambient restriction reconstructs the full assignment	`TypedEquality.Annotated.CTele.inst_ambientSub`	PQ	proved
I	Typed dependent telescope application preserves equality	`TypedEquality.Annotated.CTele.apply_equal`	PQ	proved
I	Relative telescope abstractions compute under a full independently typed assignment	`TypedEquality.Annotated.CTele.lams_apply`	PQ	proved
I	Selected annotated policy transport: opaque values	`NativeNaturalVectorFamilies.Programs.Annotated.opaque_values`	PCQ	proved
I	Selected annotated policy transport: presentation	`NativeNaturalVectorFamilies.Programs.Annotated.presentation`	PCQ	proved
I	Selected annotated policy transport: permissions morphism	`NativeNaturalVectorFamilies.Programs.Annotated.permissions_morphism`	PCQ	proved
I	Selected annotated policy transport: judgment mono	`NativeNaturalVectorFamilies.Programs.Annotated.judgment_mono`	PCQ	proved
I	Selected annotated policy transport: context mono	`NativeNaturalVectorFamilies.Programs.Annotated.context_mono`	PCQ	proved
I	Selected annotated policy transport: substitution mono	`NativeNaturalVectorFamilies.Programs.Annotated.substitution_mono`	PCQ	proved
I	Selected annotated policy transport: erase root	`NativeNaturalVectorFamilies.Programs.Annotated.erase_root`	PCQ	proved
I	Selected annotated policy transport: natural patterns linear	`NativeNaturalVectorFamilies.Programs.Annotated.natural_patterns_linear`	PCQ	proved
I	Selected annotated policy transport: vector cons repeats parameter	`NativeNaturalVectorFamilies.Programs.Annotated.vector_cons_repeats_parameter`	P	proved
I	Selected annotated policy transport: naturalOnly included	`NativeNaturalVectorFamilies.Programs.Annotated.Append.naturalOnly_included`	PCQ	proved
I	Selected annotated policy transport: function after two extensions	`NativeNaturalVectorFamilies.Programs.Annotated.Append.function_after_two_extensions`	PCQ	proved
I	Selected annotated policy transport: nil before cons	`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_before_cons`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Selected annotated policy transport: cons certificate strict	`NativeNaturalVectorFamilies.Programs.Annotated.Append.cons_certificate_strict`	PCQ	proved
I	Selected annotated policy transport: all profileVerdicts	`NativeNaturalVectorFamilies.Programs.Annotated.Append.all_profileVerdicts`	PCQ	proved
I	Premise translation commutes with head-map composition	`TypedEquality.Annotated.CPremise.mapHead_comp`	P	proved
I	Compose premise-preserving annotated package morphisms	`TypedEquality.Annotated.ChurchRules.Morphism.comp`	P	proved
I	Transport a presented annotated package under an enlarged schema family	`TypedEquality.Annotated.ChurchRules.ofSchemas_morphism`	PQ	proved
I	Typed natural fact interface: fullDomains	`HOL.NaturalFacts.fullDomains`	none	proved
I	Typed natural fact interface: recursion meaning	`HOL.NaturalFacts.recursion_meaning`	none	proved
I	Typed natural fact interface: induction meaning	`HOL.NaturalFacts.induction_meaning`	none	proved
I	Typed natural fact interface: noConfusion meaning	`HOL.NaturalFacts.noConfusion_meaning`	none	proved
I	Typed natural fact interface: successorInjective meaning	`HOL.NaturalFacts.successorInjective_meaning`	none	proved
I	Typed natural fact interface: infinity meaning	`HOL.NaturalFacts.infinity_meaning`	none	proved
I	Typed natural fact interface: collapsing noConfusion refused	`HOL.NaturalFacts.collapsing_noConfusion_refused`	P	proved
I	Typed natural fact interface: encodeTerm rename	`HOLLight.HOLSyntaxWire.encodeTerm_rename`	PQ	proved
I	Typed natural fact interface: names distinct	`HOLLight.NaturalFacts.names_distinct`	none	proved
I	Typed natural fact interface: binder names distinct	`HOLLight.NaturalFacts.binder_names_distinct`	P	proved
I	Typed natural fact interface: bare zero statement distinct	`HOLLight.NaturalFacts.bare_zero_statement_distinct`	P	proved
I	Authored carrier at the native fact signature: models recursion	`IndexedInductiveNaturalHOL.NativeFacts.models_recursion`	PQ	proved
I	Authored carrier at the native fact signature: models induction	`IndexedInductiveNaturalHOL.NativeFacts.models_induction`	PQ	proved
I	Authored carrier at the native fact signature: models noConfusion	`IndexedInductiveNaturalHOL.NativeFacts.models_noConfusion`	PQ	proved
I	Authored carrier at the native fact signature: models successorInjective	`IndexedInductiveNaturalHOL.NativeFacts.models_successorInjective`	PQ	proved
I	Authored carrier at the native fact signature: models infinity	`IndexedInductiveNaturalHOL.NativeFacts.models_infinity`	PQ	proved
I	Authored carrier at the native fact signature: all models	`IndexedInductiveNaturalHOL.NativeFacts.all_models`	PQ	proved
I	Authored carrier at the native fact signature: boolean recursion witness	`IndexedInductiveNaturalHOL.NativeFacts.boolean_recursion_witness`	P	proved
I	Observed telescope retains a typed application head	`FormationSensitive.Typing.applyClosed_head`	P	proved
I	Observed telescope recovers its original declaration spine	`FormationSensitive.DeclarationSpine.ofObservedTelescope`	P	proved
I	A declaration head retains its closed formation	`FormationSensitive.DeclarationSpine.constant_formed`	P	proved

Part	Claim	Lean declaration	Axioms	Status
I	Original vector eliminator declaration	`ComputationProfiles.NaturalConversion.Preservation.Vector.recursor_declared`	PCQ	proved
I	Vector telescope application is the original eliminator call	`ComputationProfiles.NaturalConversion.Preservation.Vector.application_shape`	P	proved
I	Vector telescope recovers the dependent principal result	`ComputationProfiles.NaturalConversion.Preservation.Vector.principal_result`	P	proved
I	Observed vector application retains its original declaration	`ComputationProfiles.NaturalConversion.Preservation.Vector.recursor_spine`	PCQ	proved
I	All dependent vector arguments and displayed-type replay	`ComputationProfiles.NaturalConversion.Preservation.Vector.recover`	PCQ	proved
I	Operational vector nil preserves smaller-policy typing	`ComputationProfiles.NaturalConversion.Preservation.Vector.nil_preserves`	PCQ	proved
I	Original vector cons declaration	`ComputationProfiles.NaturalConversion.Preservation.Vector.constructor_declared`	PCQ	proved
I	Observed constructor recovers its four dependent arguments	`ComputationProfiles.NaturalConversion.Preservation.Vector.constructor_arguments`	PCQ	proved
I	Canonical vector cons target from original eliminator formation	`ComputationProfiles.NaturalConversion.Preservation.Vector.canonical_cons_target`	PCQ	proved
I	Operational vector cons preserves smaller-policy typing	`ComputationProfiles.NaturalConversion.Preservation.Vector.cons_preserves`	PCQ	proved
I	All original operational roots preserve smaller checking judgments	`ComputationProfiles.NaturalConversion.Preservation.operational_roots`	PCQ	proved
I	Existing producer yields the canonical nil submission	`ComputationProfiles.NaturalConversion.Controls.vector_nil_produced`	PCQ	proved
I	Existing producer yields the canonical cons submission	`ComputationProfiles.NaturalConversion.Controls.vector_cons_produced`	PCQ	proved
I	Canonical vector nil source checks under natural-only policy	`ComputationProfiles.NaturalConversion.Controls.vector_nil_typed`	PCQ	proved
I	Canonical vector cons source checks under natural-only policy	`ComputationProfiles.NaturalConversion.Controls.vector_cons_typed`	PCQ	proved
I	Canonical operational nil target retains its type	`ComputationProfiles.NaturalConversion.Controls.vector_nil_preserved`	PCQ	proved
I	Canonical operational cons target retains its type	`ComputationProfiles.NaturalConversion.Controls.vector_cons_preserved`	PCQ	proved
I	Operational cons is absent from smaller checking roots	`ComputationProfiles.NaturalConversion.Controls.vector_cons_root_disabled`	PCQ	proved
I	Smaller checker refuses the operational cons citation	`ComputationProfiles.NaturalConversion.Controls.vector_cons_citation_refused`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Full checker accepts the exact same cons citation	`ComputationProfiles.NaturalConversion.Controls.vector_cons_citation_enabled`	PCQ	proved
I	Actual original declaration roots preserve smaller checking judgments	`ComputationProfiles.NaturalConversion.Preservation.operational_rule_roots`	PCQ	proved
I	Actual closure parameters form an admitted Henkin valuation	`HOL.NaturalClosure.parameterValuation_admissible`	none	proved
I	Actual lambda denotes the strengthened closure predicate	`HOL.NaturalClosure.denote_strengthening`	none	proved
I	Henkin term closure admits the particular induction predicate	`HOL.NaturalClosure.strengthening_admitted`	none	proved
I	Closure-premised induction from local parameter admission	`HOL.NaturalClosure.closedUnder_induction_admitted`	none	proved
I	Guarded induction holds in every selected Henkin model	`HOL.NaturalClosure.models_induction_henkin`	none	proved
I	Three-point interpretation has exactly its generated zero	`HOL.NaturalClosure.InductionControls.closed_iff_zero`	P	proved
I	Three-point guarded step holds	`HOL.NaturalClosure.InductionControls.guarded_step`	P	proved
I	Three-point unguarded step fails	`HOL.NaturalClosure.InductionControls.unguarded_step_refused`	P	proved
I	Three-point actual guarded induction	`HOL.NaturalClosure.InductionControls.guarded_induction`	P	proved
I	Native guarded-step and strengthened-definition controls	`MM0.NaturalInduction.all_verdicts`	P	proved
I	Common cumulative head preservation for annotated packages	`Annotated.CHeadPreserving.ofLevelTower`	P	proved
I	Original disabled table has no raw root	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_raw_no_root`	PCQ	proved
I	Actual declaration policies reuse the tower head reading	`NativeNaturalVectorFamilies.Programs.Annotated.headReading`	PCQ	proved
I	Actual declaration policies retain ground-head equality	`NativeNaturalVectorFamilies.Programs.Annotated.ground_head_eq`	PCQ	proved
I	Opaque annotated declarations have former injectivity and no-confusion	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_formerFacts`	PCQ	proved
I	Opaque annotated product cannot equal a universe head	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_pi_not_head`	PCQ	proved
I	Cumulative heads preserve typing under every actual permission selection	`NativeNaturalVectorFamilies.Programs.Annotated.head_preserving`	PCQ	proved
I	Opaque contextual reductions are equal at the original dependent type	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_reduction_equal`	PCQ	proved
I	Opaque contextual reductions retain the original dependent type	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_reduction_typed`	PCQ	proved
I	Actual opaque natural function product is not a universe	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_function_type_not_universe`	PCQ	proved
I	Actual opaque natural function takes a typed beta step	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_function_beta`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Opaque declarations refuse the actual natural zero citation	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_zero_citation_refused`	PCQ	proved
I	Enabling rows accepts that same natural zero citation	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.full_zero_citation_accepted`	PCQ	proved
M1	Nontrivial interpretation of the original natural type	`NaturalDomain.reading_nat`	PCQ	proved
M1	Original zero is the numeral zero ideal	`NaturalDomain.reading_zero`	PCQ	proved
M1	Original successor uses the projected numeral constructor	`NaturalDomain.reading_successor`	PCQ	proved
M1	Original recursor uses the projected dependent recursor	`NaturalDomain.reading_recursor`	PCQ	proved
M1	Actual natural declaration matches the numeral interface	`NaturalDomain.nat_declared`	PCQ	proved
M1	Actual zero declaration matches the numeral interface	`NaturalDomain.zero_declared`	PCQ	proved
M1	Actual successor declaration matches the numeral interface	`NaturalDomain.successor_declared`	PCQ	proved
M1	Actual dependent recursor declaration matches the numeral interface	`NaturalDomain.recursor_declared`	PCQ	proved
M1	Every original constant is fixed by its declared projection	`NaturalDomain.constants_valid`	PCQ	proved
M1	Original recursor lookup recovers the semantic dependent spine	`NaturalDomain.recursor_spine`	PCQ	proved
M1	Natural zero computation with original semantic spine premises	`NaturalDomain.zero_valid`	PCQ	proved
M1	Natural successor computation with original semantic spine premises	`NaturalDomain.successor_valid`	PCQ	proved
M1	Both actual selected annotated rows have valid interpretations	`NaturalDomain.roots_valid`	PCQ	proved
M1	Nontrivial valid reading of the natural-only annotation	`NaturalDomain.valid`	PCQ	proved
M1	Existing actual addition proof has equal denotations	`NaturalDomain.one_plus_zero`	PCQ	proved
M1	Actual successor of zero denotes its numeral ideal	`NaturalDomain.successor_zero`	PCQ	proved
M1	Actual one plus zero denotes successor of zero	`NaturalDomain.one_plus_zero_value`	PCQ	proved
M1	The natural interpretation separates zero and successor	`NaturalDomain.zero_ne_successor_zero`	PCQ	proved
M1	Soundness refutes the false annotated constructor equation	`NaturalDomain.zero_equal_successor_refused`	PCQ	proved
M1	Larger declaration spine facts retain original types	`SpineFacts.of_decl_inclusion`	PQ	proved
M1	Semantic validity of an opaque package sum retaining computation	`ReadingValid.sum_opaque`	PQ	proved
M1	Decoder proposition name is fresh for the original signature	`NaturalDomain.Decoder.fresh_prop`	PCQ	proved
M1	Decoder name is fresh for the original signature	`NaturalDomain.Decoder.fresh_holds`	PCQ	proved
M1	Opaque proposition declaration has its original type	`NaturalDomain.Decoder.prop_declared`	PCQ	proved
M1	Opaque decoder declaration has its original type	`NaturalDomain.Decoder.holds_declared`	PCQ	proved
M1	Original nontrivial reading validates opaque decoder declarations	`NaturalDomain.Decoder.declarations_valid`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
M1	Semantic validity with original natural computation and fresh decoder	`NaturalDomain.Decoder.valid`	PCQ	proved
M1	Decoder extension retains exactly the original natural root cases	`NaturalDomain.Decoder.roots_iff`	PCQ	proved
M1	Decoder extension retains original root evidence requirements	`NaturalDomain.Decoder.premises_iff`	PCQ	proved
M1	Actual original typed addition proof persists into decoder extension	`NaturalDomain.Decoder.one_plus_zero`	PCQ	proved
M1	Actual natural zero computation remains enabled	`NaturalDomain.Decoder.zero_root`	PCQ	proved
M1	Decoder extension still refutes the false constructor equation	`NaturalDomain.Decoder.zero_equal_successor_refused`	PCQ	proved
M1	Abstraction/representation roundtrip has the defined type	`HOLTypeDefinitionBoundary.roundtrip_typed`	PQ	proved
M1	Equality at the defined type passes source typing	`HOLTypeDefinitionBoundary.defined_equality_typed`	PQ	proved
M1	Distinct representation equality index is refused	`HOLTypeDefinitionBoundary.representation_equality_refused`	PQ	proved
M1	Correctly indexed equation passes source ingress	`HOLTypeDefinitionBoundary.defined_equality_checked`	PCQ	proved
M1	Wrong equality index refuses source ingress	`HOLTypeDefinitionBoundary.representation_equality_check_refused`	PCQ	proved
M1	Concrete representation and defined types differ	`HOLTypeDefinitionBoundary.distinct_types`	none	proved
M1	Type-definition boundary controls compute	`HOLTypeDefinitionBoundary.all_verdicts`	PQ	proved
M1	Constant expansion retains the complete schema-premise list	`patternPremises_instConsts_of_fixed`	PQ	proved
M1	Supported schema roots survive constant expansion	`SchemaConstantsFixed.root`	PQ	proved
M1	Required evidence follows the same constant expansion	`SchemaConstantsFixed.requires`	PQ	proved
M1	Supported declarations and schemas give a full constant interpretation	`ChurchRules.ofSchemas_constInstance`	PQ	proved
M1	Fresh proposition expands to its universe	`NaturalDomain.Decoder.interpret_prop`	PQ	proved
M1	Fresh decoder expands to the universe identity	`NaturalDomain.Decoder.interpret_holds`	PQ	proved
M1	Every original declared name is fixed	`NaturalDomain.Decoder.original_names_fixed`	PCQ	proved
M1	All eight original declaration types pass finite support checking	`NaturalDomain.Decoder.declarationTypeFixedCheck_true`	PCQ	proved
M1	Every actual declared type is fixed by the return map	`NaturalDomain.Decoder.original_types_fixed`	PCQ	proved
M1	All four original left templates are fixed	`NaturalDomain.Decoder.row_left_fixed`	PCQ	proved
M1	All four original right templates are fixed	`NaturalDomain.Decoder.row_right_fixed`	PCQ	proved
M1	All four rows retain their inferred variable types	`NaturalDomain.Decoder.row_variable_type_fixed`	PCQ	proved
M1	Every selected original row has full constant-interpretation support	`NaturalDomain.Decoder.schema_support`	PCQ	proved
M1	Original computation and evidence survive the interpretation	`NaturalDomain.Decoder.original_interpretation`	PCQ	proved
M1	Decoder type expansion reuses the maintained definitions	`NaturalDomain.Decoder.decoder_type_expansion`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
M1	Full return interpretation for every original permission selection	`NaturalDomain.Decoder.return_interpretation`	PCQ	proved
M1	Return preserves complete judgments and contexts	`NaturalDomain.Decoder.returned_judgment`	PCQ	proved
M1	Equal derivability for statements fixed by the return interpretation	`NaturalDomain.Decoder.fixed_statement_iff`	PCQ	proved
M1	The fresh proposition is typed in each extension	`NaturalDomain.Decoder.proposition_typed`	PCQ	proved
M1	Its changed universe image is typed in the original package	`NaturalDomain.Decoder.returned_proposition_typed`	PCQ	proved
M1	Fresh proposition typing is not fixed by the return map	`NaturalDomain.Decoder.proposition_statement_changes`	PQ	proved
M1	The actual original addition statement is fixed	`NaturalDomain.Decoder.original_addition_fixed`	PQ	proved
M1	Actual natural-only addition reflects through the return route	`NaturalDomain.Decoder.original_addition_reflected`	PCQ	proved
M1	Actual full-policy addition reflects through the return route	`NaturalDomain.Decoder.full_policy_addition_reflected`	PCQ	proved
M1	Return interpretation refutes the false natural constructor equation	`NaturalDomain.Decoder.false_equation_refused_by_return`	PCQ	proved
M1	Legacy identity typing omits domain formation	`FormationSensitive.raw_identity_at_any_domain`	P	proved
M1	Refined identity typing retains its domain declaration	`FormationSensitive.judgment_identity_domain_declared`	PQ	proved
M1	Raw/refined identity boundary at a missing domain	`FormationSensitive.raw_identity_missing_domain`	PQ	proved
M1	Formed contexts alone do not upgrade every legacy derivation	`FormationSensitive.no_uniform_raw_upgrade`	PQ	proved
M1	Selected carrier no-preimage formula has the omitted-point meaning	`MM0.NaturalCarrier.noPreimage_meaning`	none	proved
M1	Selected carrier reference-format controls compute	`MM0.NaturalCarrier.all_verdicts`	P	proved
M1	Natural-subtype statement/domain controls compute	`MM0.NaturalType.all_verdicts`	P	proved
A3	Finite-stack realization of the HOL Light to MM0 translation	—	—	open
A2	Existing shared-lemma HOL translation equals the node-by-node translation	—	—	open
A4	Free symmetric monoidal category of consuming processes	—	—	open
C	Portable modules for rho, MeTTa equations, recipes, rule-code kernels	—	—	open

15 Open problems

1. **Finite-stack realization of the HOL Light to MM0 translation.** The certificate bound is proved and the declared time account is bounded below by output weight, but no finite-stack machine computing the translation is given; a pushdown transducer for planar rule images is the natural first case.
2. **The existing lemma translation.** The generic node-by-node translation commutes with unfolding; prove that the existing shared-lemma HOL Light translation, which works over cumulative contexts and raw proofs, computes the same per-node rule images.
3. **Monoidal processes for consuming steps.** Independent consuming steps commute and the split/join example exceeds tree-shaped single-wire cuts; the free symmetric monoidal category of

processes, with traces identified exactly up to independence, is not formalized.

4. **Memory of the executed MM0 compilation.** On a sixteen-event export with ten definitional roots, the existing compiler exhausted its memory allowance while compiling the last of sixteen shared proofs; witness sharing in the compiled MM0 proofs is the evident target.
5. **Remaining portable modules.** Rho, MeTTa equation and authored recipe kernels, and the propositional rule-code kernels, have faithful checkers but no portable module presentation.
6. **Admitted dependent carve-outs and native specialization.** Complete the context, substitution and theory comparisons for declared dependent fragments; give their smaller packages executable decoders and computational-admission proofs; connect the actual native checking machines through adequate operational routes. Proof-relevant route agreement, model assumptions and executed cost bounds remain separate obligations. Compare derived checking bases with the existing native kernels under the same fragment and cost model.

References

[1] Dharmakirti. *Nyayabindu*, chapters I–III, with Dharmottara's commentary. English translation in Th. Stcherbatsky, *Buddhist Logic*, vol. II, Bibliotheca Buddhica XXVI, 1930.

[2] D. J. Chalmers and F. Jackson. Conceptual analysis and reductive explanation. *The Philosophical Review* 110 (2001), 315–361, Section 2. https://consc.net/papers/analysis.html.

[3] D. J. Chalmers. *Constructing the World*. Oxford University Press, 2012, especially chapters 1–3 and the sixth excursus on totality and indexical truths. https://consc.net/constructing-the-world/.

[4] S. A. Kripke. Outline of a theory of truth. *The Journal of Philosophy* 72 (1975), 690–716.

[5] J. Lambek. Deductive systems and categories II: standard constructions and closed categories. In *Category Theory, Homology Theory and their Applications I*, Lecture Notes in Mathematics 86, Springer, 1969, 76–122.

[6] C. Williams and M. Stay. Native Type Theory. *Electronic Proceedings in Theoretical Computer Science* 372 (2022), 116–132. https://doi.org/10.4204/EPTCS.372.9.

[7] P. Saville. Clones, closed categories, and combinatory logic. *FoSSaCS 2024*, Lecture Notes in Computer Science 14575, pp. 160–181. https://arxiv.org/abs/2405.01675.

[8] M. Carneiro. Metamath Zero: the Cartesian theorem prover. Preprint, 2019. https://arxiv.org/abs/1910.10703.

[9] J. Jang, A. Gaulin, J. Z. S. Hu and B. Pientka. McTT: A verified kernel for a proof assistant. *Proceedings of the ACM on Programming Languages*, ICFP, 2025. https://doi.org/10.1145/3747511.

[10] L. Pujet and N. Tabareau. Observational equality: now for good. *Proceedings of the ACM on Programming Languages* 6, POPL, 2022. https://doi.org/10.1145/3498693.

[11] F. Blanqui, G. Dowek, E. Grienenberger, G. Hondet and F. Thiré. A modular construction of type theories. *Logical Methods in Computer Science* 19(1), 2023, 12:1–12:28. https://doi.org/10.46298/lmcs-19(1:12)2023.

[12] M. Hofmann. *Extensional Concepts in Intensional Type Theory*. PhD thesis, University of Edinburgh, 1995. https://hdl.handle.net/1842/399.

[13] K. Kapulkin and Y. Li. Extensional concepts in intensional type theory, revisited. *Theoretical Computer Science* 1029, 2025, 115051. https://arxiv.org/abs/2310.05706.